

UNIT-IV

The Transport Layer

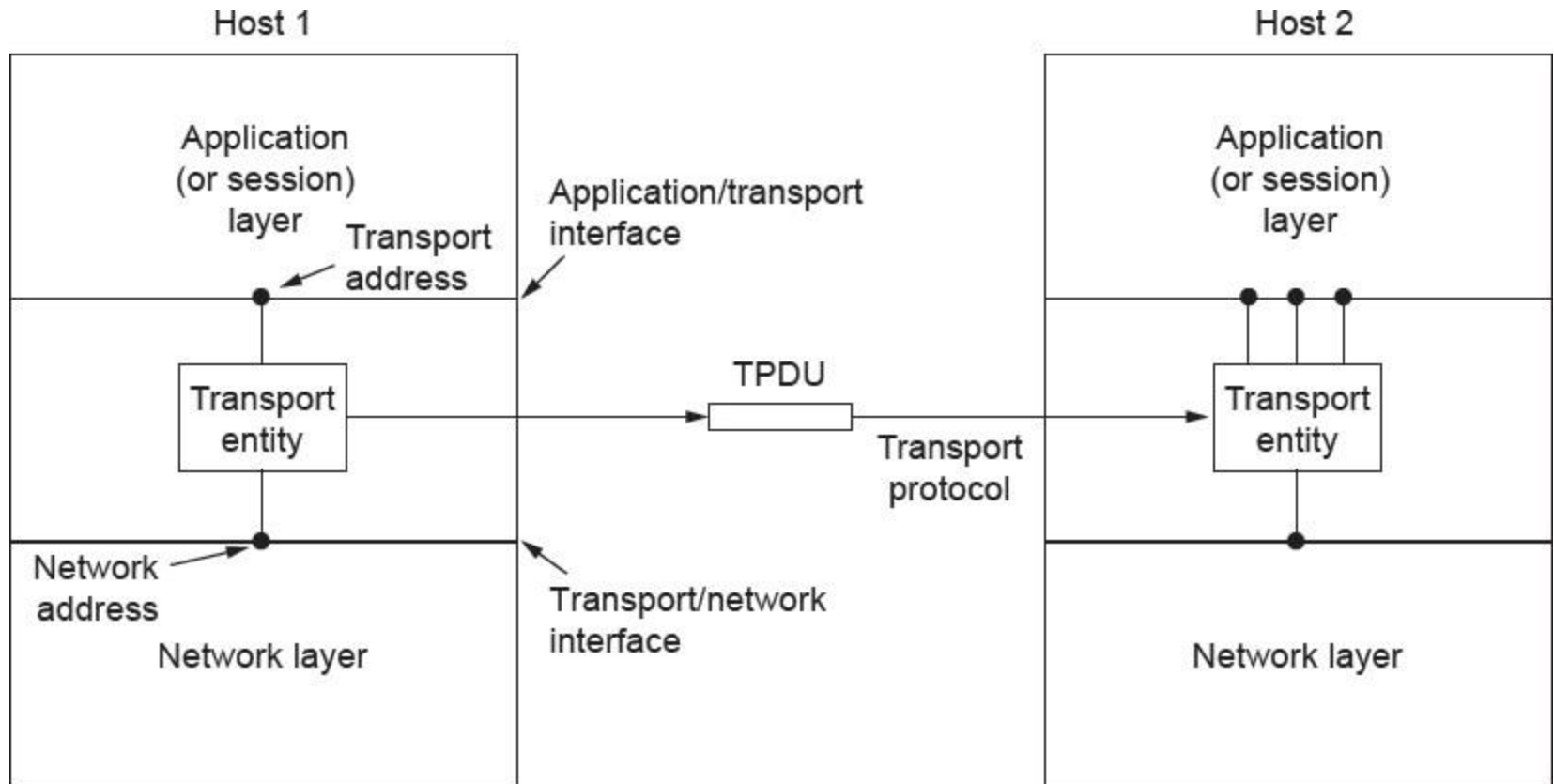
Transport Service

- Upper Layer Services
- Transport Service Primitives
- Berkeley Sockets
- Example of Socket Programming:
Internet File Server

The Transport Layer

To allow users to access the transport service, the transport layer must provide some operations to application programs, that is, a transport service interface.

Services Provided to the Upper Layers



The network, transport, and application layers

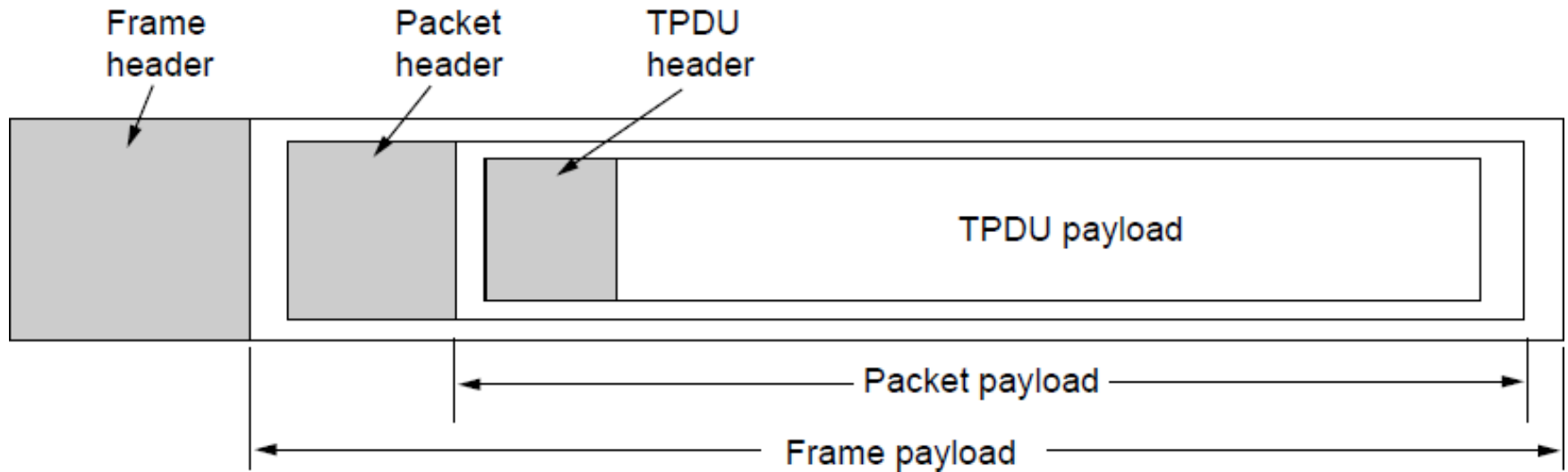
Establishment, data transfer and release/ Make it more reliable than the

Transport Service Primitives (1)

Primitive	Packet sent	Meaning
LISTEN	(none)	Block until some process tries to connect
CONNECT	CONNECTION REQ.	Actively attempt to establish a connection
SEND	DATA	Send information
RECEIVE	(none)	Block until a DATA packet arrives
DISCONNECT	DISCONNECTION REQ.	This side wants to release the connection

The primitives for a simple transport service

Transport Service Primitives (2)



Nesting of TPDU (Transport Protocol Data Unit) s, packets, and frames.

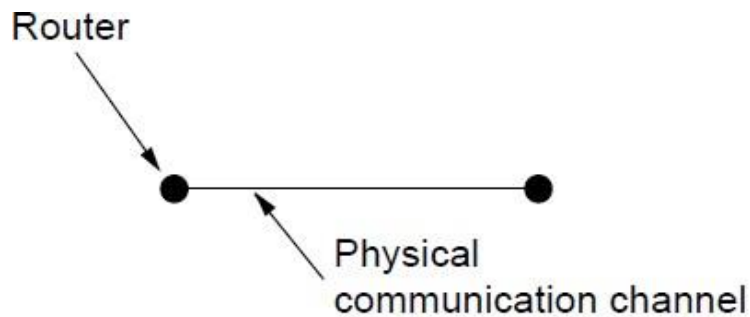
Elements of Transport Protocols (1)

- Addressing
- Connection establishment
- Connection release
- Error control and flow control
- Multiplexing
- Crash recovery

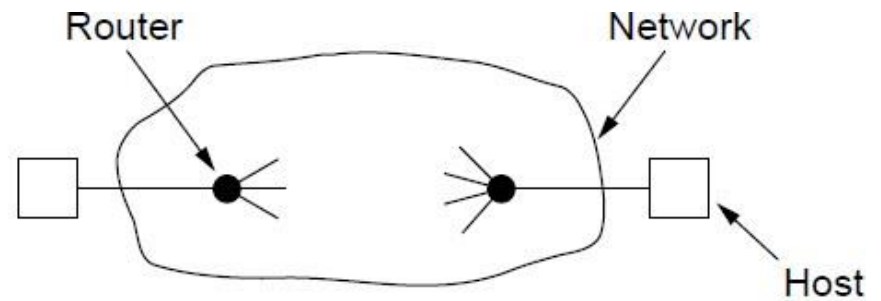
Similarity between data link and transport layer

- Connection establishment
- Connection release
- Error control and flow control

Elements of Transport Protocols (2)



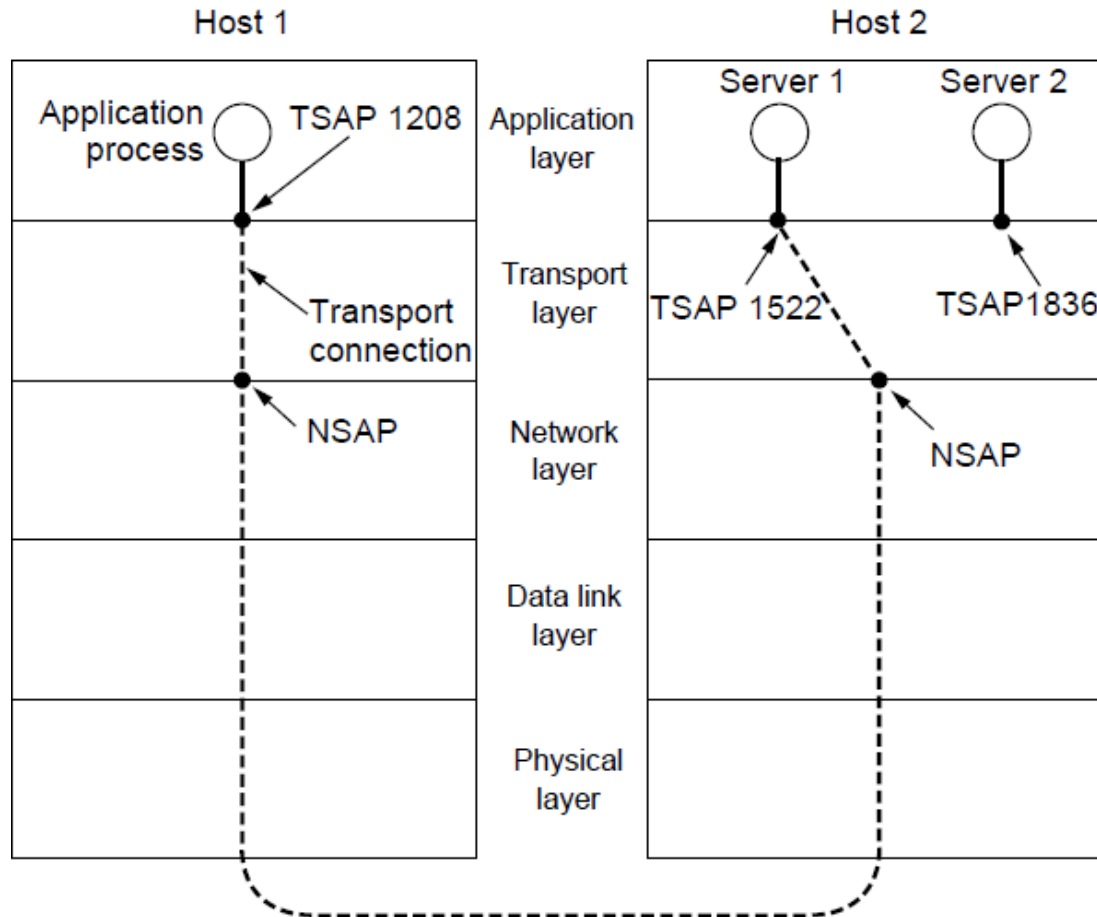
(a)



(b)

- (a) Environment of the data link layer.
- (b) Environment of the transport layer.

Addressing (1)



TSAPs, NSAPs, and transport connections

Addressing (2)

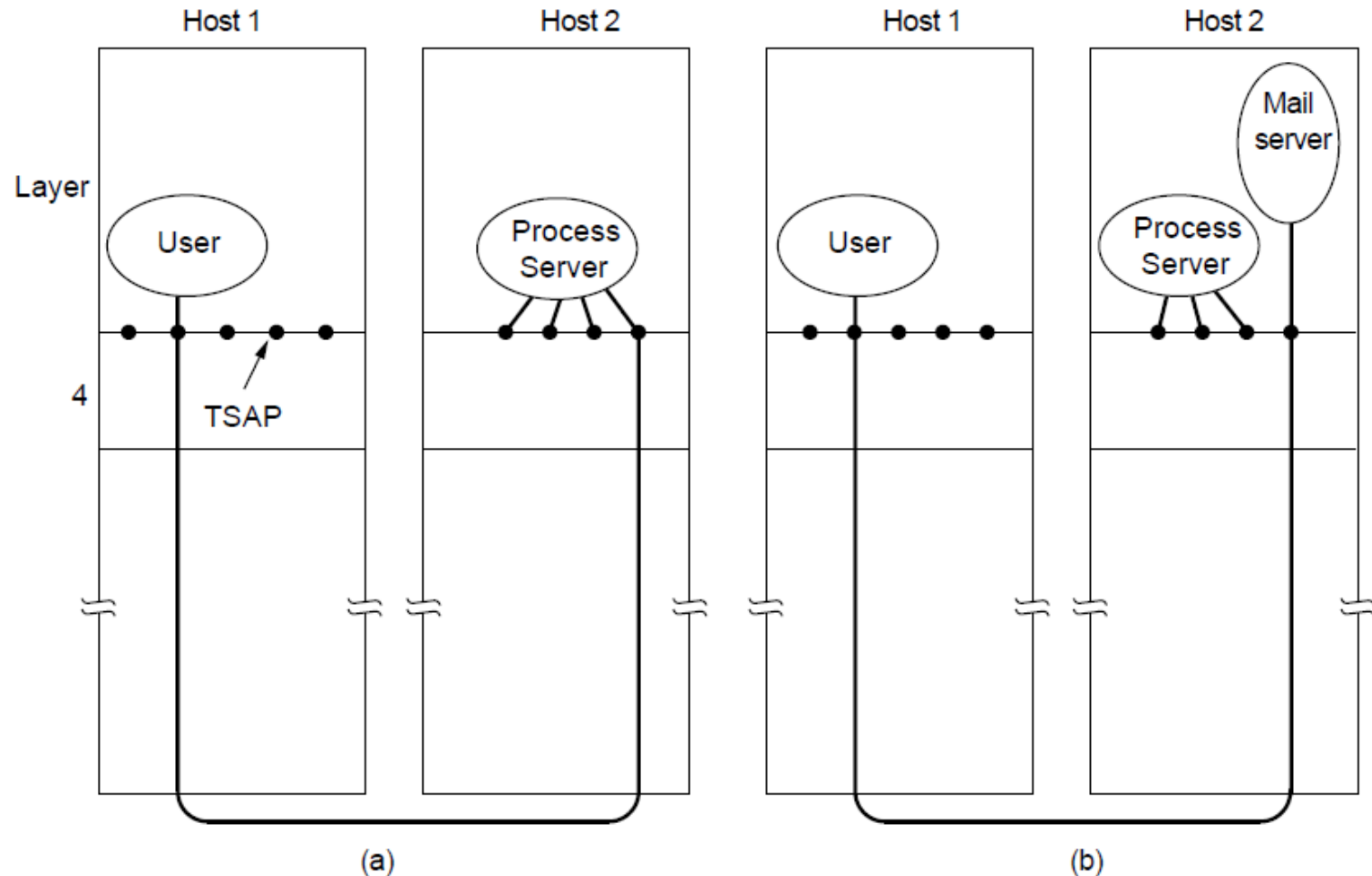
A possible scenario for a transport connection is as follows:

1. A mail server process attaches itself to TSAP 1522 on host 2 to wait for an incoming call. How a process attaches itself to a TSAP is outside the networking model and depends entirely on the local operating system. A call such as our LISTEN might be used, for example.
2. An application process on host 1 wants to send an email message, so it attaches itself to TSAP 1208 and issues a CONNECT request. The request specifies TSAP 1208 on host 1 as the source and TSAP 1522 on host 2 as the destination. This action ultimately results in a transport connection being established between the application process and the server.

Addressing (3)

3. The application process sends over the mail message.
4. The mail server responds to say that it will deliver the message.
5. The transport connection is released.

Addressing (4)



How a user process in host 1 establishes a connection with a mail server in host 2 via a process server.

Connection Establishment (1)

Techniques for restricting packet lifetime

A. Throwaway transport addresses

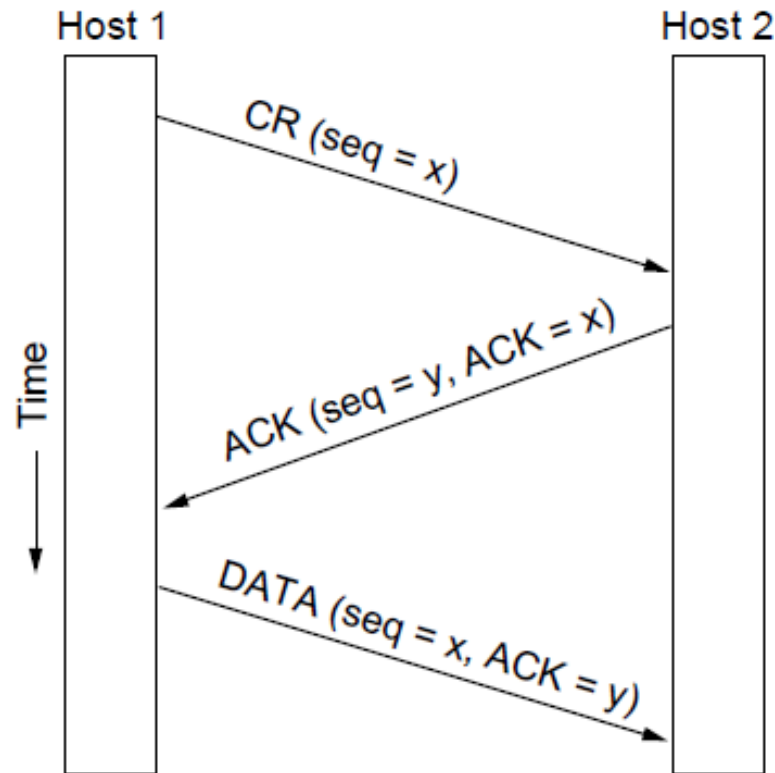
B. Each connection a identifier <Peer transport entity, connection entity>

Connection Establishment (2)

Maintain history problem if crashed

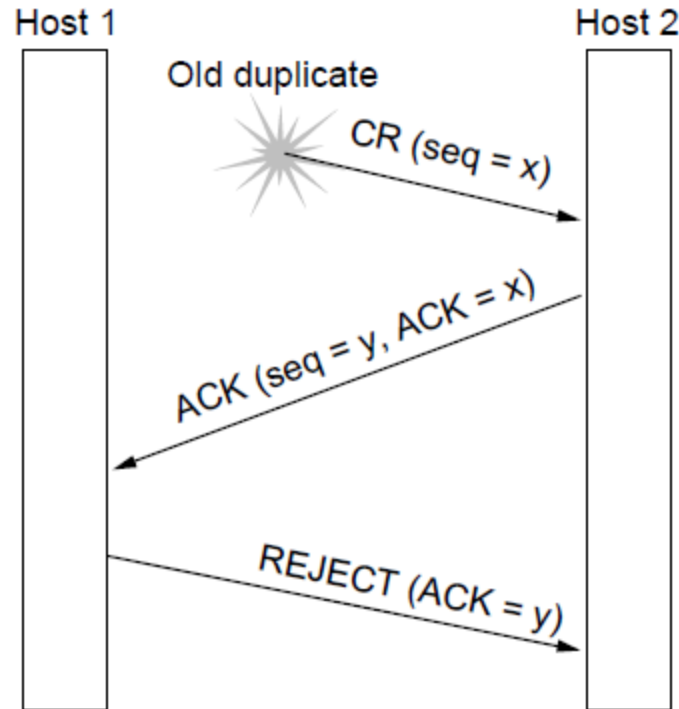
- Restricted network design.
 - Putting a hop counter in each packet.
 - Timestamping each packet.
-
- Also acknowledgement needs to be erased with time T

Connection Establishment (3)



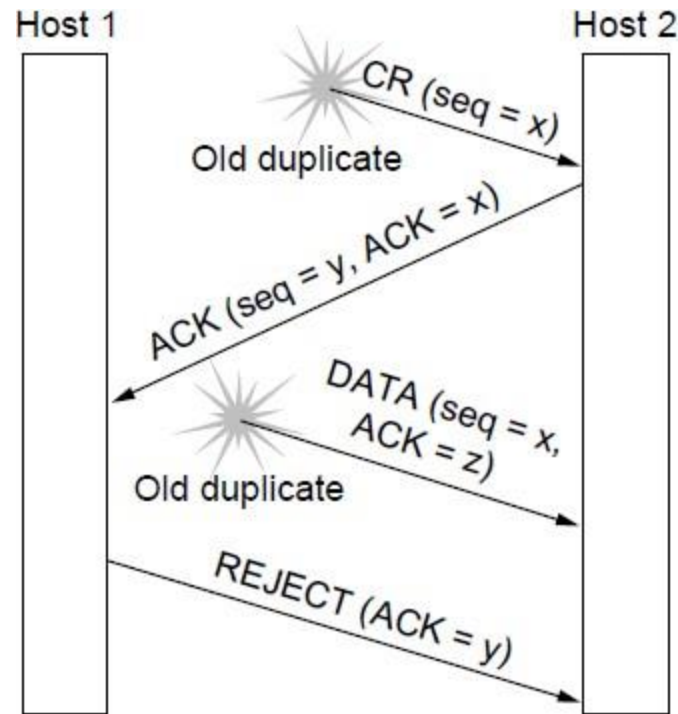
Three protocol scenarios for establishing a connection using a three-way handshake. CR denotes CONNECTION REQUEST.

Connection Establishment (4)



Three protocol scenarios for establishing a connection using a three-way handshake. CR denotes CONNECTION REQUEST. Old duplicate CONNECTION REQUEST appearing out of nowhere.

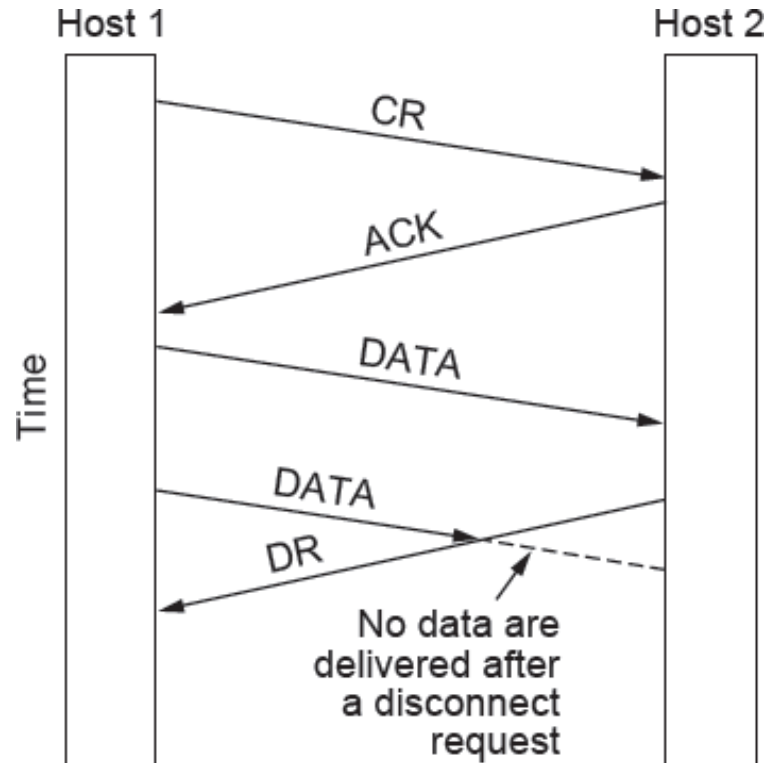
Connection Establishment (5)



Three protocol scenarios for establishing a connection using a three-way handshake. CR denotes CONNECTION REQUEST.

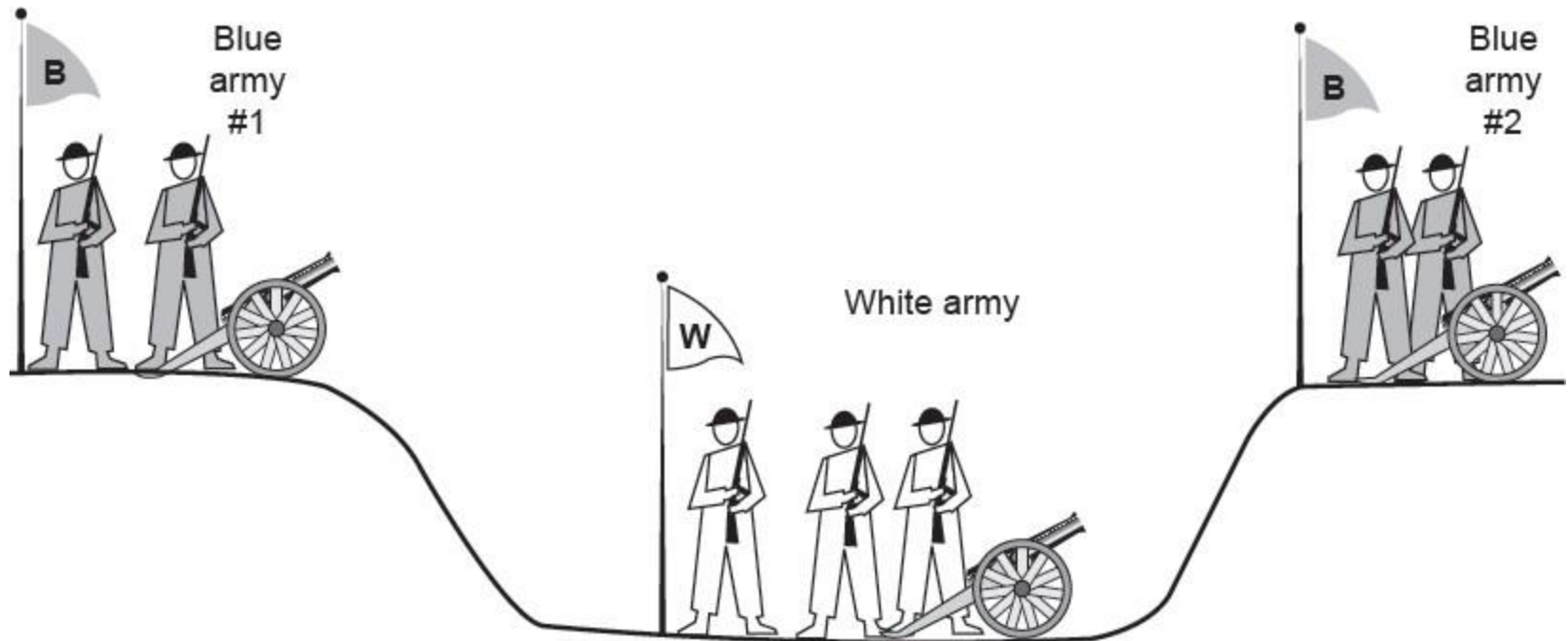
Duplicate CONNECTION REQUEST and duplicate ACK

Connection Release (1)



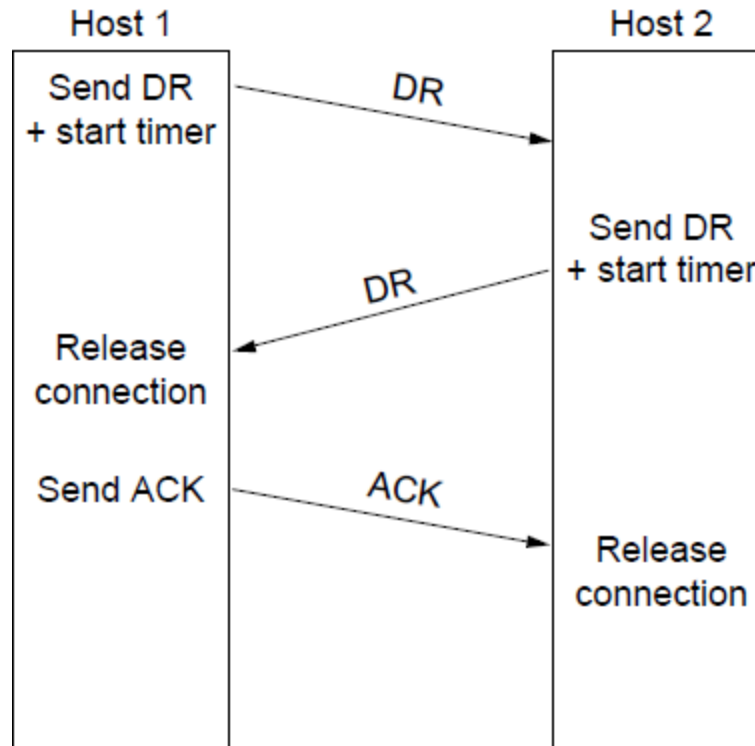
Abrupt disconnection with loss of data – one way release or two way release

Connection Release (2)



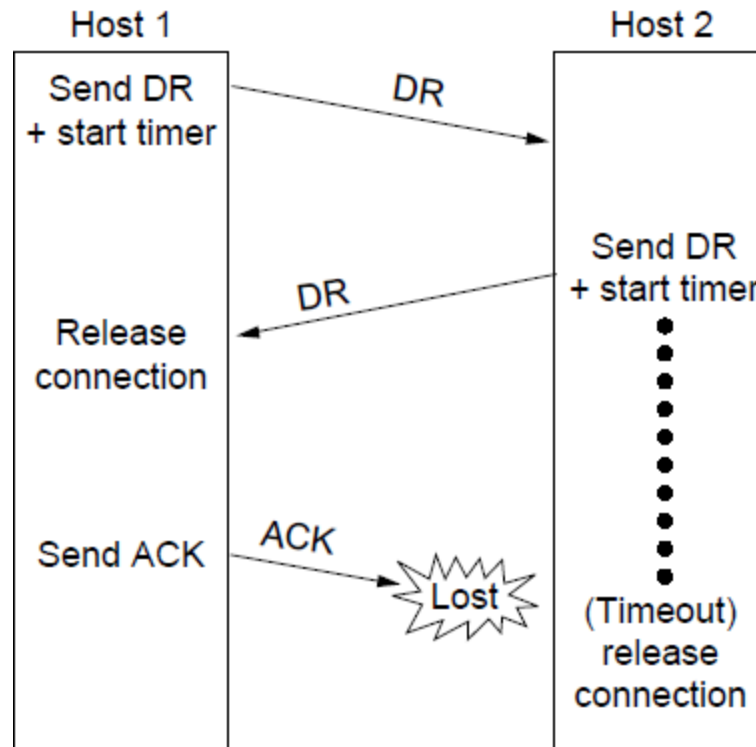
The two-army problem
Synchronize which will go on
infinitely

Connection Release (3)



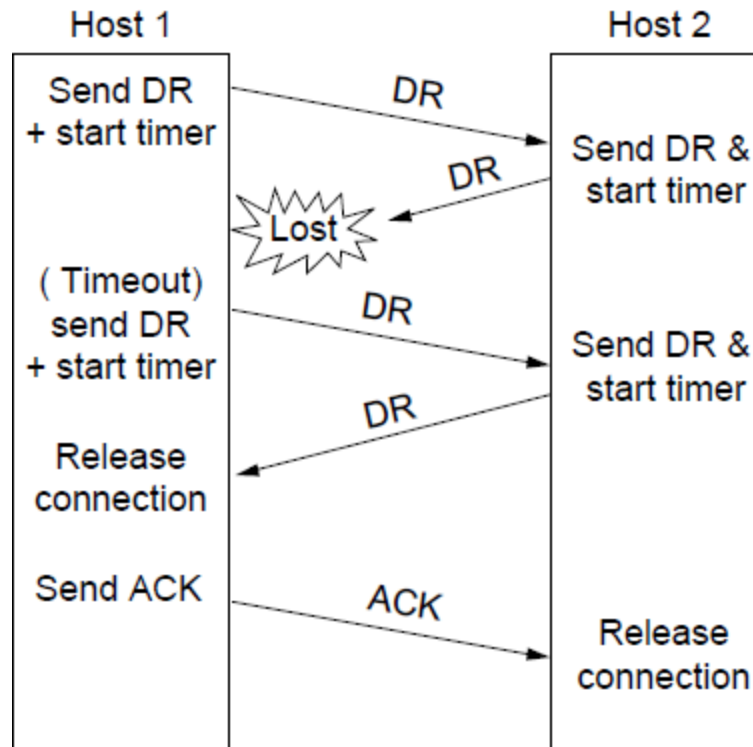
Four protocol scenarios for releasing a connection.
(a) Normal case of three-way handshake

Connection Release (4)



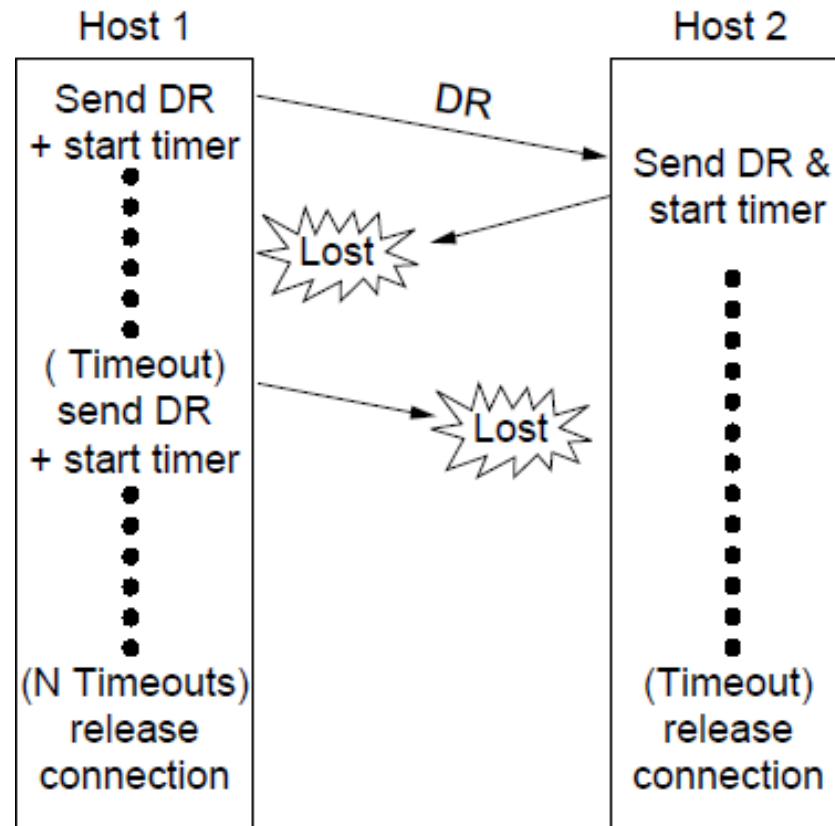
Four protocol scenarios for releasing a connection.
(b) Final ACK lost.

Connection Release (5)



Four protocol scenarios for releasing a connection.
(c) Response lost

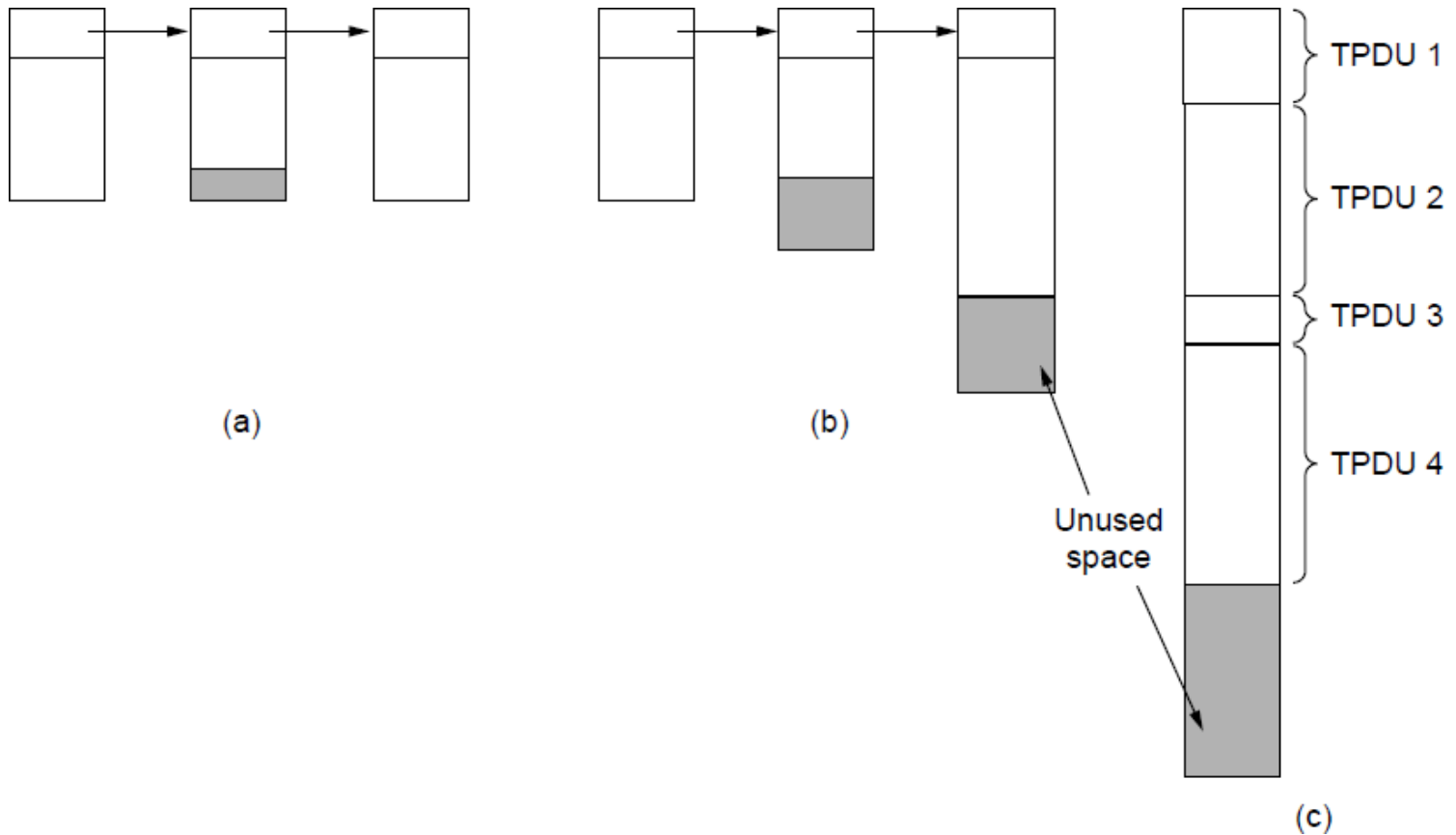
Connection Release (6)



Error Control and Flow Control (1)

- ✓ Error control is ensuring that the data is delivered with the desired level of reliability, usually that all of the data is delivered without any errors.
- ✓ Flow control is keeping a fast transmitter from overrunning a slow receiver.

Error Control and Flow Control (2)



(a) Chained fixed-size buffers. (b) Chained variable-sized buffers. (c) One large circular buffer per connection.

Error Control and Flow Control (3)

As a very brief recap:

1. A frame carries an error-detecting code (e.g., a CRC or checksum) that is used to check if the information was correctly received.
2. A frame carries a sequence number to identify itself and is retransmitted by the sender until it receives an acknowledgement of successful receipt from the receiver. This is called ARQ (Automatic Repeat reQuest).
3. There is a maximum number of frames that the sender will allow to be outstanding at any time, pausing if the receiver is not acknowledging frames quickly enough. If this maximum is one packet the protocol is called stop-and-wait. Larger windows enable pipelining and improve performance on long, fast links.
4. The sliding window protocol combines these features and is also used to support bidirectional data transfer.

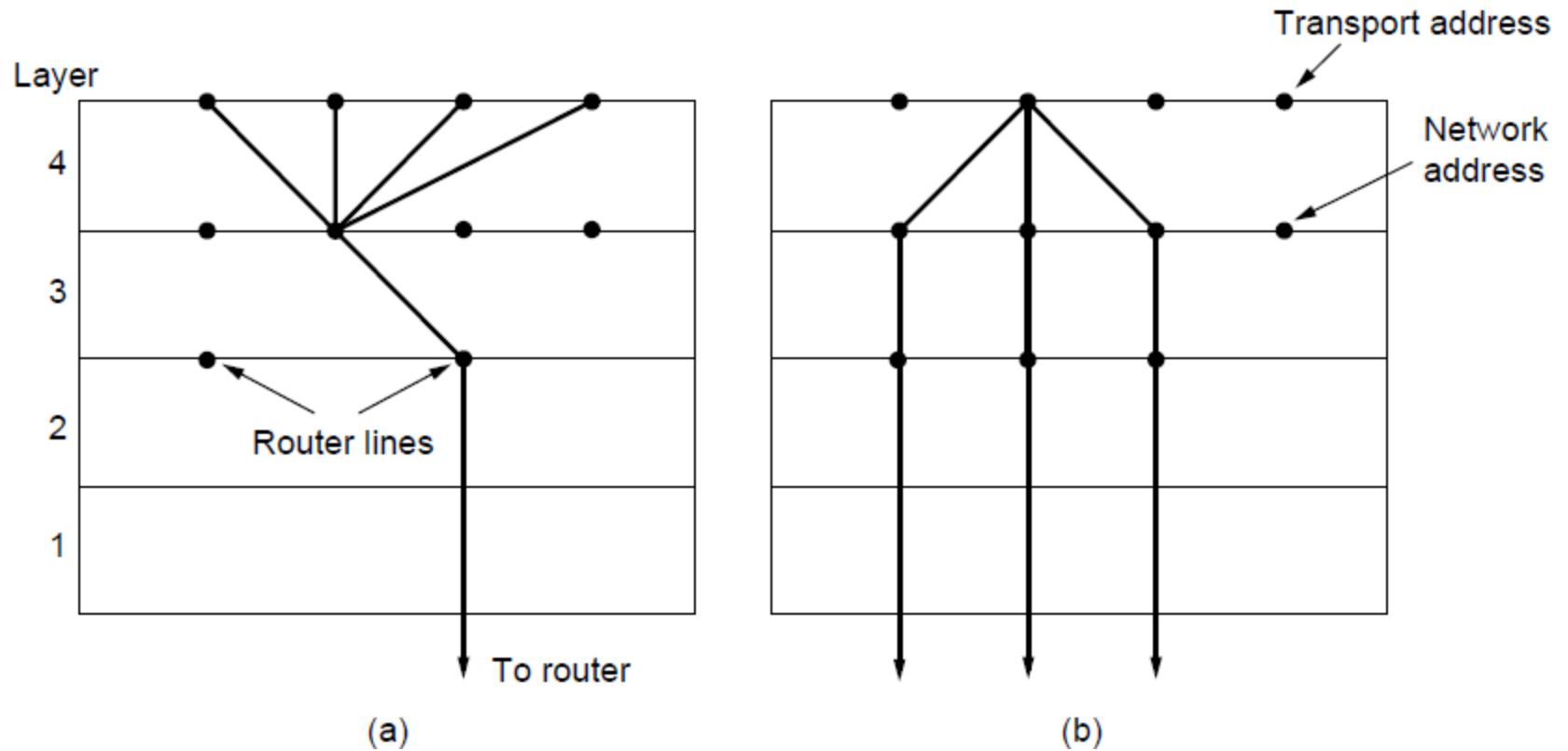
- a) For low-bandwidth bursty traffic, it is better not allot buffer but dynamically allot buffer
- b) Decouple sliding window protoco;

Error Control and Flow Control (2)

	<u>A</u>	<u>Message</u>	<u>B</u>	<u>Comments</u>
1	→	< request 8 buffers>	→	A wants 8 buffers
2	←	<ack = 15, buf = 4>	←	B grants messages 0-3 only
3	→	<seq = 0, data = m0>	→	A has 3 buffers left now
4	→	<seq = 1, data = m1>	→	A has 2 buffers left now
5	→	<seq = 2, data = m2>	...	Message lost but A thinks it has 1 left
6	←	<ack = 1, buf = 3>	←	B acknowledges 0 and 1, permits 2-4
7	→	<seq = 3, data = m3>	→	A has 1 buffer left
8	→	<seq = 4, data = m4>	→	A has 0 buffers left, and must stop
9	→	<seq = 2, data = m2>	→	A times out and retransmits
10	←	<ack = 4, buf = 0>	←	Everything acknowledged, but A still blocked
11	←	<ack = 4, buf = 1>	←	A may now send 5
12	←	<ack = 4, buf = 2>	←	B found a new buffer somewhere
13	→	<seq = 5, data = m5>	→	A has 1 buffer left
14	→	<seq = 6, data = m6>	→	A is now blocked again
15	←	<ack = 6, buf = 0>	←	A is still blocked
16	...	<ack = 6, buf = 4>	←	Potential deadlock

Dynamic buffer allocation. The arrows show the direction of transmission. An ellipsis (...) indicates a lost TPDU

Multiplexing



(a) Multiplexing. (b) Inverse multiplexing.

Crash Recovery

No matter how the client and server are programmed, there are always situations where the protocol fails to recover properly.

The server can be programmed in one of two ways:
acknowledge first or write first.

The client can be programmed in one of four ways:
always retransmit the last segment, never retransmit the last segment, retransmit only in state $S0$, retransmit only in state $S1$.

This gives eight combinations.

Server side events

Three events are possible at the server:
sending an acknowledgement (A), writing to the output process (W),
and crashing (C).

The three events can occur in six different orderings: AC(W), AWC, C(AW), C(WA), WAC, and WC(A), where the parentheses are used to indicate that **neither A nor W** can follow C (i.e., once it has crashed, it has crashed).

Client Side Events

The client can be programmed in one of four ways:

- ✓ always retransmit the last segment,
- ✓ never retransmit the last segment,
- ✓ retransmit only in state S0,
- ✓ retransmit only in state S1.

Each client can be in one of two states:

- ☐ one segment outstanding, S1, or
- ☐ no segments outstanding, S0

Crash Recovery

Strategy used by sending host	Strategy used by receiving host					
	First ACK, then write			First write, then ACK		
	AC(W)	AWC	C(AW)	C(WA)	W AC	WC(A)
Always retransmit	OK	DUP	OK	OK	DUP	DUP
Never retransmit	LOST	OK	LOST	LOST	OK	OK
Retransmit in S0	OK	DUP	LOST	LOST	DUP	OK
Retransmit in S1	LOST	OK	OK	OK	OK	DUP

OK = Protocol functions correctly
 DUP = Protocol generates a duplicate message
 LOST = Protocol loses a message

Different combinations of client and server strategy

The Internet Transport Protocols: TCP (1)

- Introduction to TCP
- The TCP service model
- The TCP protocol
- The TCP segment header
- TCP connection establishment
- TCP connection release

The Internet Transport Protocols: TCP (2)

- TCP connection management modeling
- TCP sliding window
- TCP timer management
- TCP congestion control
- TCP futures

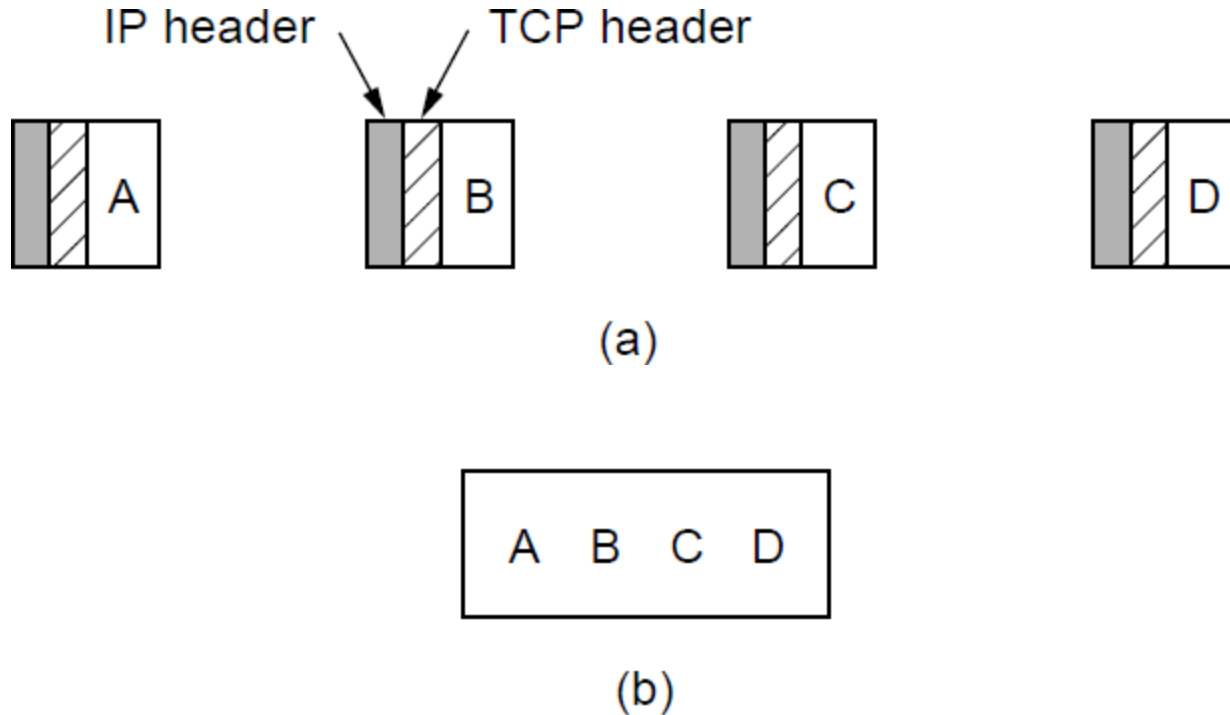
The TCP Service Model (1)

Port	Protocol	Use
20, 21	FTP	File transfer
22	SSH	Remote login, replacement for Telnet
25	SMTP	Email
80	HTTP	World Wide Web
110	POP-3	Remote email access
143	IMAP	Remote email access
443	HTTPS	Secure Web (HTTP over SSL/TLS)
543	RTSP	Media player control
631	IPP	Printer sharing

Some assigned ports

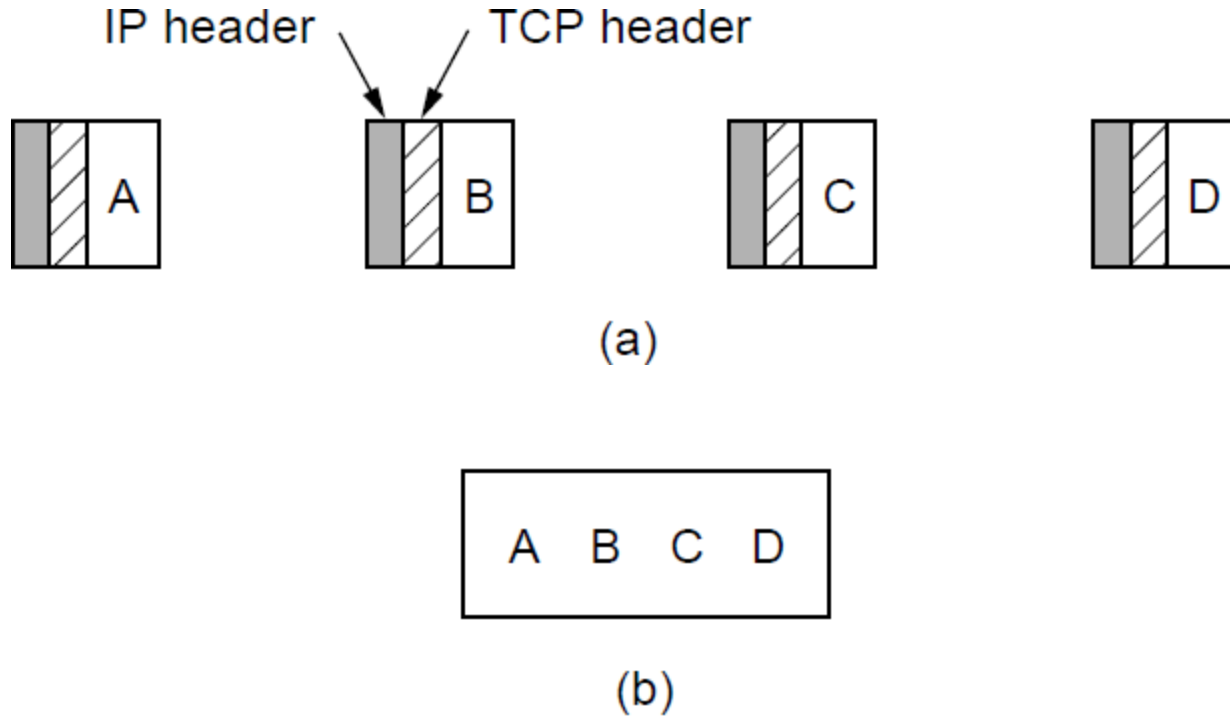
Internet Daemon (inetd) attach itself to multiple ports and wait for the first connection request, then fork to that service

The TCP Service Model (2)



- (a) The TCP is byte-stream and not a message-stream, so messages are not differentiated.
- (b) Four 512-byte segments sent as separate IP datagrams
- (c) The 2048 bytes of data delivered to the application in a single READ call

The TCP Service Model (2)



(a) To force data out -- PUSH flag

(b) Too many PUSH-es then all PUSH are collected together and sent.

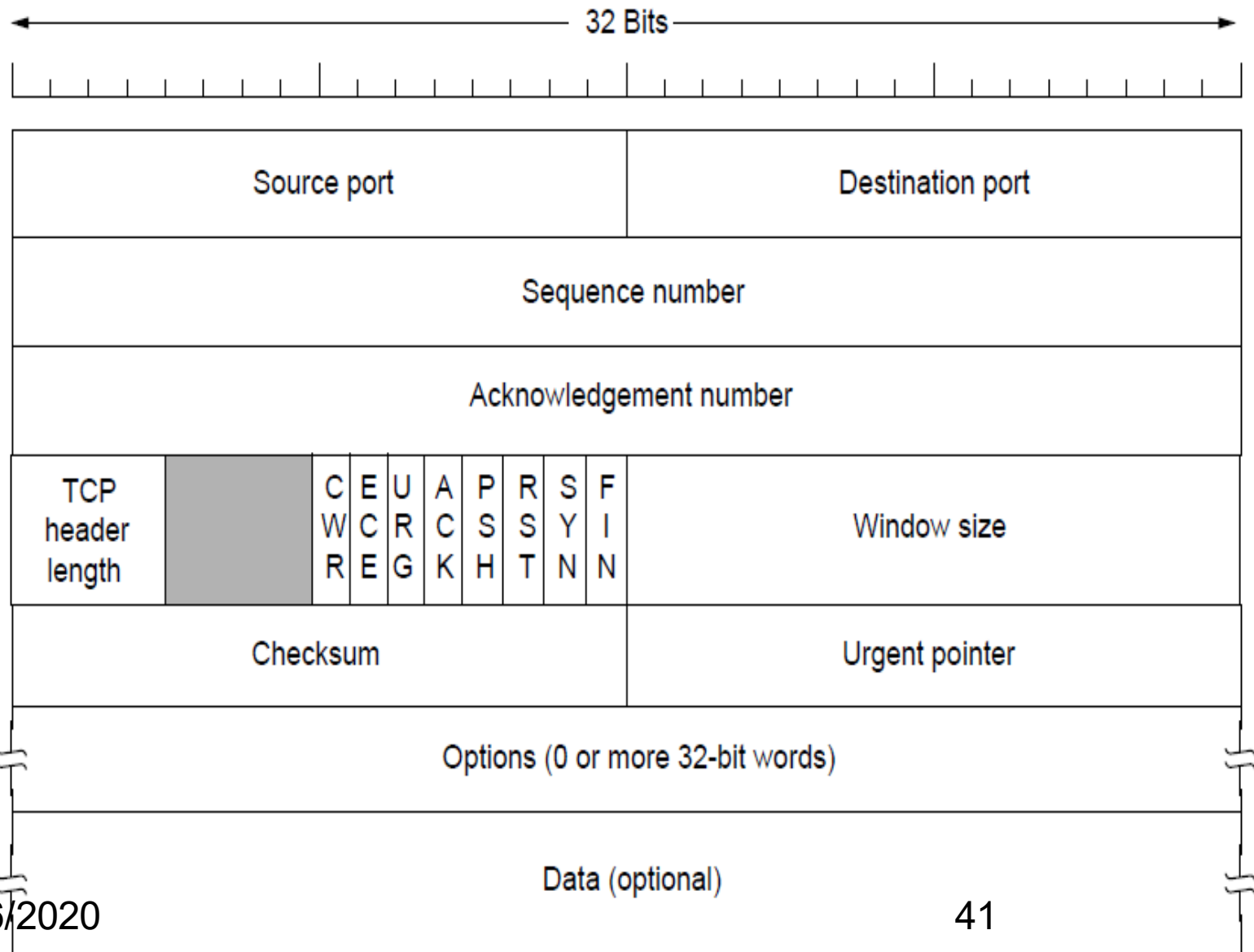
(c) URGENT – on pushing Ctrl-C to break-off remote

4c/6o/2m⁰²p⁰utation, the sending application puts s³o⁹me control flag

TCP Header

TCP segment – 20 byte IP header, 20 –byte TCP header, total
 $65,535 = 64 \text{ KB}$

TCP Header



TCP Header

1. Source Port-

Source Port is a 16 bit field.

It identifies the port of the sending application.

2. Destination Port-

Destination Port is a 16 bit field.

It identifies the port of the receiving application.

A TCP connection is uniquely identified by using-
Combination of port numbers and IP Addresses of sender and receiver
IP Addresses indicate which systems are communicating.

Port numbers indicate which end to end sockets are communicating

TCP Header

3. Sequence Number-

Sequence number is a 32 bit field.

TCP assigns a unique sequence number to each byte of data contained in the TCP segment.

This field contains the sequence number of the first data byte.

4. Acknowledgement Number-

Acknowledgment number is a 32 bit field.

It contains sequence number of the data byte that receiver expects to receive next from the sender.

It is always sequence number of the last received data byte incremented by 1.

TCP Header

5. Header Length-

Header length is a 4 bit field.

It contains the length of TCP header.

It helps in knowing from where the actual data begins.

The length of TCP header always lies in the range-

— [20 bytes , 60 bytes]

The initial 5 rows of the TCP header are always used.

So, minimum length of TCP header = $5 \times 4 \text{ bytes} = 20 \text{ bytes}$.

The size of the 6th row representing the Options field vary.

The size of Options field can go up to 40 bytes.

So, maximum length of TCP header = $20 \text{ bytes} + 40 \text{ bytes} = 60 \text{ bytes}$

TCP Header

6. Reserved Bits-

The 6 bits are reserved. These bits are not used. 7.FLAGS

7.1 CWR & ECE-- *CWR* and *ECE* are used to signal congestion
CWR(*Congestion Window Reduced*)

ECE(*Explicit Congestion Notification-Echo*)

--*ECE* is set to signal an *ECN-Echo* to a TCP sender to tell it to slow down when the TCP receiver gets a congestion indication from the network.

--*CWR* is set to signal *Congestion Window Reduced* from the TCP sender to the TCP receiver so that it knows the sender has slowed down and can stop sending the *ECN-Echo*.

TCP Header

7.2.URG –Bit

URG bit is used to treat certain data on an urgent basis. When URG bit is set to 1,

- It indicates the receiver that certain amount of data within the current segment is urgent.
- Urgent data is pointed out by evaluating the urgent pointer field.
- The urgent data has be prioritized.
- Receiver forwards urgent data to the receiving application on a separate channel.

TCP Header

7.3.ACK –Bit

ACK bit indicates whether acknowledgement number field is valid or not.

--When ACK bit is set to 1, it indicates that acknowledgement number contained in the TCP header is valid.

--For all TCP segments except request segment, ACK bit is set to 1.

--Request segment is sent for connection establishment during Three Way Handshake.

TCP Header

7.4.PSH –Bit

PSH bit is used to push the entire buffer immediately to the receiving application.

When PSH bit is set to 1,

- All the segments in the buffer are immediately pushed to the receiving application.

- No wait is done for filling the entire buffer.

- This makes the entire buffer to free up immediately.

TCP Header

7.5.RST Bit- RST bit is used to reset the TCP connection.

When RST bit is set to 1,

- It indicates the receiver to terminate the connection immediately.
- It causes both the sides to release the connection and all its resources abnormally.
- The transfer of data ceases in both the directions.
- It may result in the loss of data that is in transit.

This is used only when-

- There are unrecoverable errors.
- There is no chance of terminating the TCP connection normally.

TCP Header

8. Window Size-

Window size is a 16 bit field.

- It contains the size of the receiving window of the sender.
- It advertises how much data (in bytes) the sender can receive without acknowledgement.
- Thus, window size is used for Flow Control.

9. Checksum-

Checksum is a 16 bit field used for error control.

- It verifies the integrity of data in the TCP payload.
- Sender adds CRC checksum to the checksum field before sending the data.
- Receiver rejects the data that fails the CRC check.

TCP Header

10. Urgent Pointer-

Urgent pointer is a 16 bit field.

--It indicates how much data in the current segment counting from the first data byte is urgent.

--Urgent pointer added to the sequence number indicates the end of urgent data byte.

--This field is considered valid and evaluated only if the URG bit is set to 1.

TCP Header

10. Urgent Pointer-

Urgent pointer is a 16 bit field.

--It indicates how much data in the current segment counting from the first data byte is urgent.

--Urgent pointer added to the sequence number indicates the end of urgent data byte.

--This field is considered valid and evaluated only if the URG bit is set to 1.

TCP Header

11. Options-

Options field is used for several purposes.

--The size of options field vary from 0 bytes to 40 bytes.

Options field is generally used for the following purposes-

1. Time stamp
2. Window size extension
3. Parameter negotiation
4. Padding

TCP Header

1. Time Stamp-

When wrap around time is less than life time of a segment,

- Multiple segments having the same sequence number may appear at the receiver side.
- This makes it difficult for the receiver to identify the correct segment.
- If time stamp is used, it marks the age of TCP segments.
- Based on the time stamp, receiver can identify the correct segment.

TCP Header

2. Window Size Extension-

Options field may be used to represent a window size greater than 16 bits.

--Using window size field of TCP header, window size of only 16 bits can be represented.

--If the receiver wants to receive more data, it can advertise its greater window size using this field.

--The extra bits are then appended in Options field.

TCP Header

3. Parameter Negotiation-

Options field is used for parameters negotiation. Example-

During connection establishment,

--Both sender and receiver have to specify their maximum segment size.

--To specify maximum segment size, there is no special field.

--So, they specify their maximum segment size using this field and negotiates.

TCP Header

3. Parameter Negotiation-

Options field is used for parameters negotiation. Example-

During connection establishment,

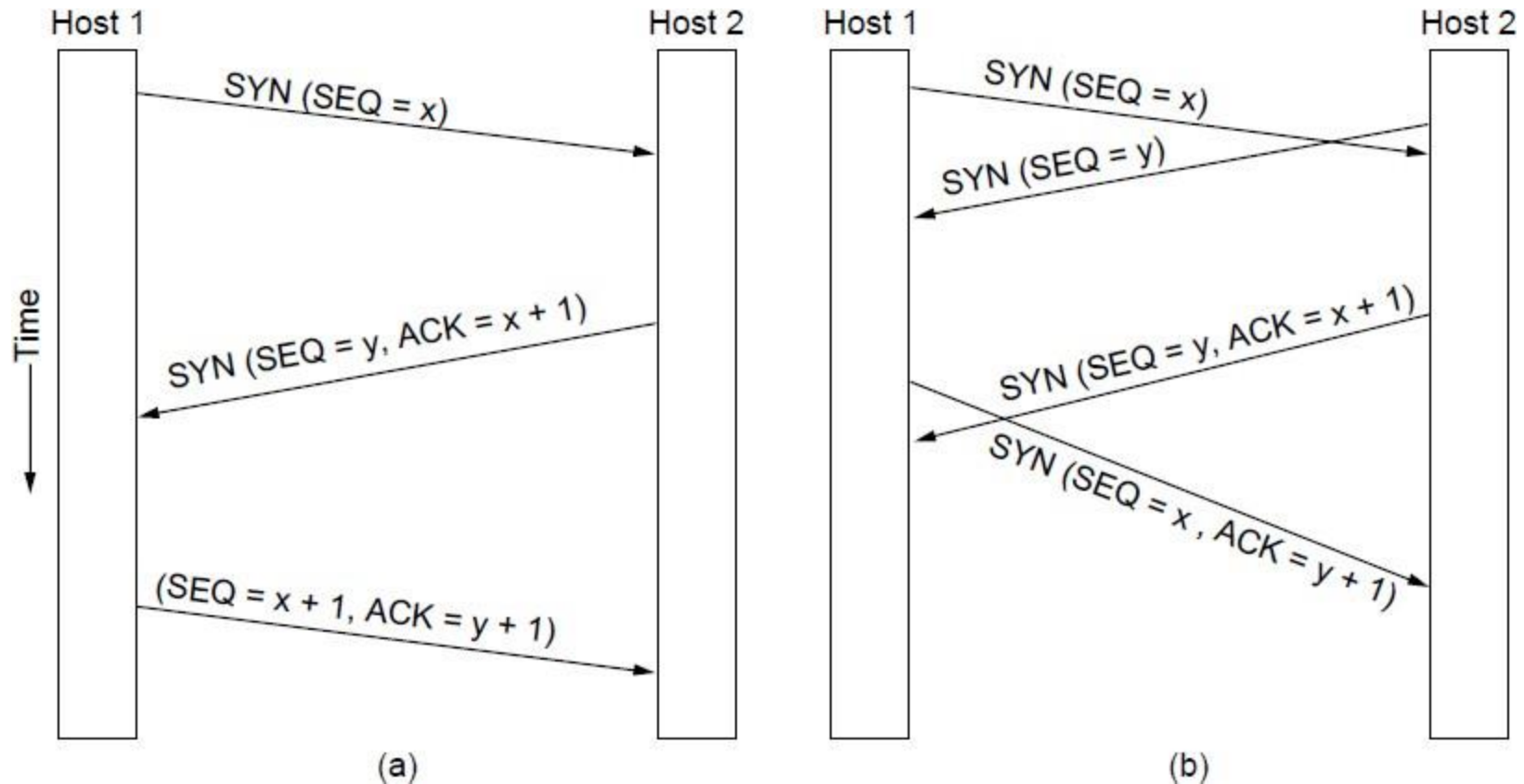
- Both sender and receiver have to specify their maximum segment size.
- To specify maximum segment size, there is no special field.
- So, they specify their maximum segment size using this and field negotiates.

4. Padding-

Addition of dummy data to fill up unused space in the transmission unit and make it conform to the standard size is called as padding.

Options field is used for padding.

TCP Connection Establishment



(a) TCP connection establishment in the normal case.

(b) Simultaneous connection establishment on both sides

TCP Connection Release

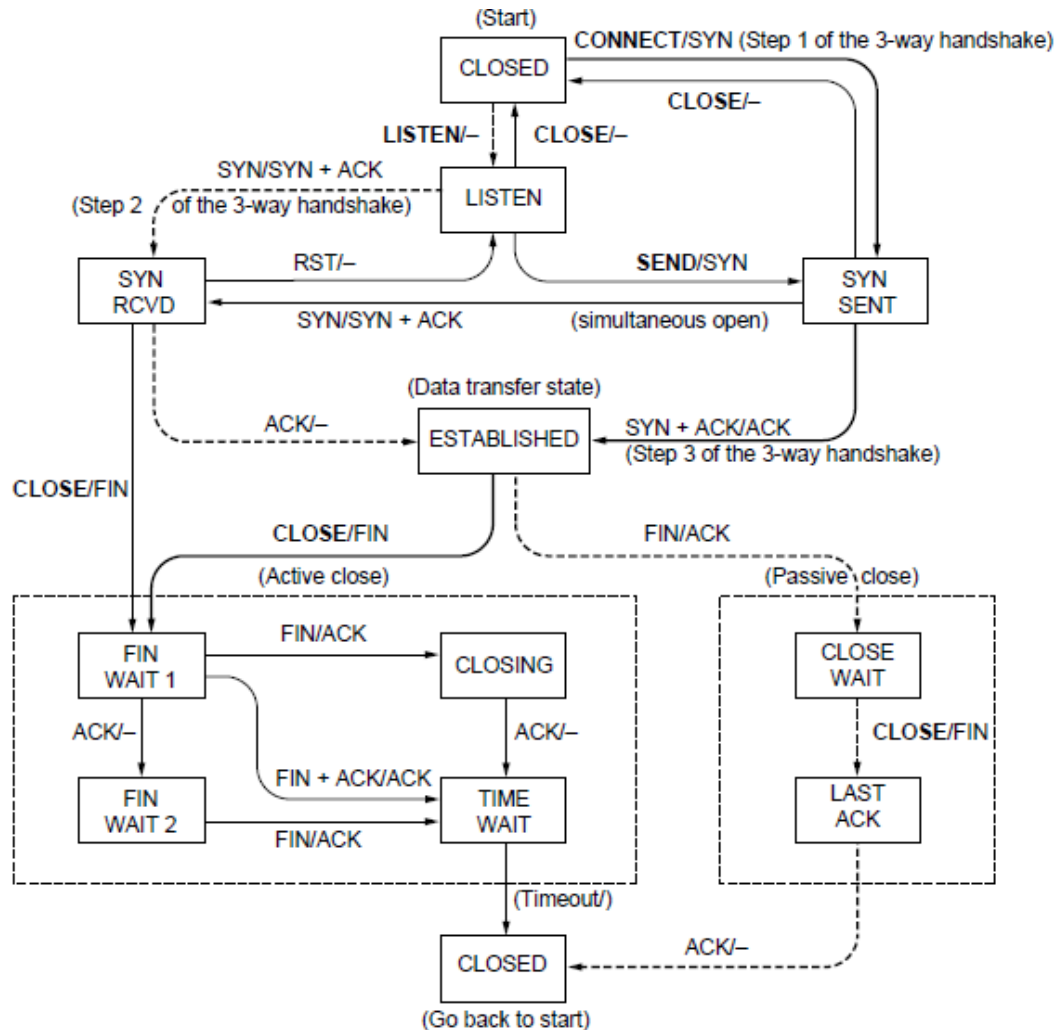
- a) Either party send with the FIN bit set
- b) When the FIN is acknowledged, that direction is shut down for new data
- c) Full closing (TWO FIN and TWO ACK)

TCP Connection Management Modeling (1)

State	Description
CLOSED	No connection is active or pending
LISTEN	The server is waiting for an incoming call
SYN RCVD	A connection request has arrived; wait for ACK
SYN SENT	The application has started to open a connection
ESTABLISHED	The normal data transfer state
FIN WAIT 1	The application has said it is finished
FIN WAIT 2	The other side has agreed to release
TIME WAIT	Wait for all packets to die off
CLOSING	Both sides have tried to close simultaneously
CLOSE WAIT	The other side has initiated a release
LAST ACK	Wait for all packets to die off

The states used in the TCP connection management finite state machine₆₀.

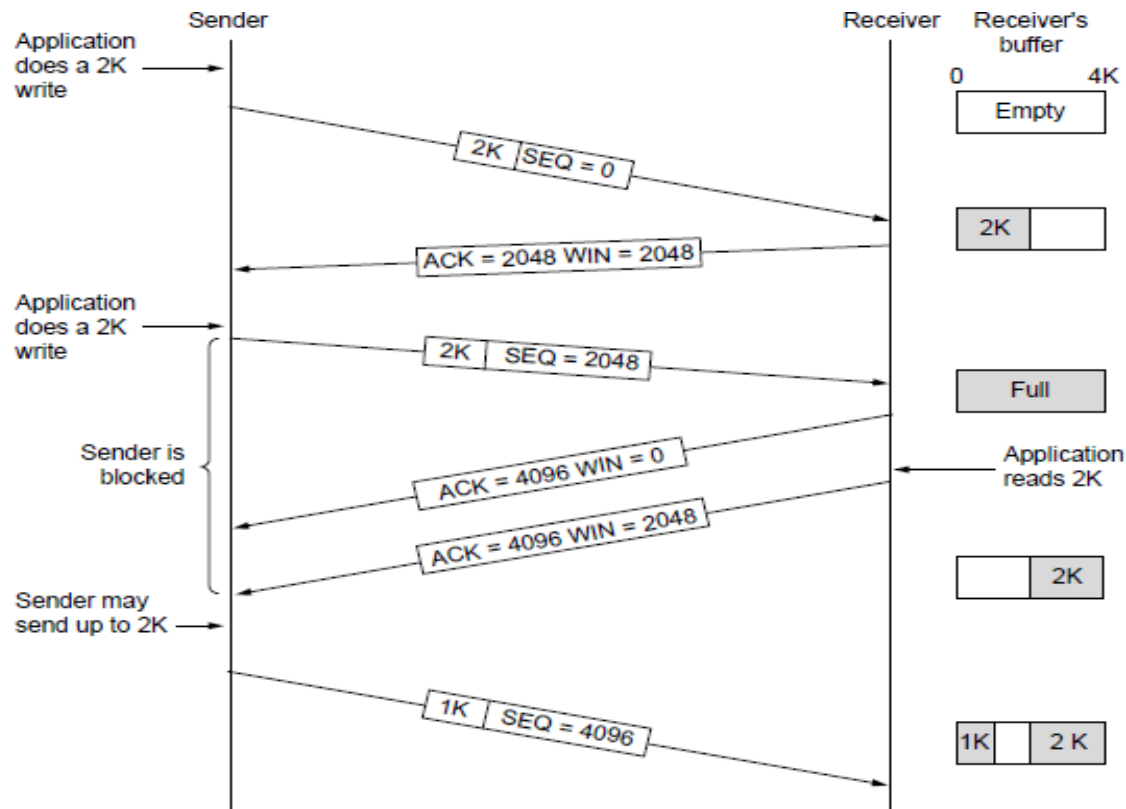
TCP Connection Management Modeling (2)



TCP connection management finite state machine.

The heavy solid line is the normal path for a client. The heavy dashed line is the normal path for a server. The light lines are unusual events. Each transition is labeled by the event causing it and the action resulting from it, separated by a slash.

TCP Sliding Window



Window management in TCP

When send – with) window size – a). Urgent bytes and b). 1- byte to make the receiver to reannounce the next byte expected

When the window is 0, the sender may not normally send segments, with two exceptions.

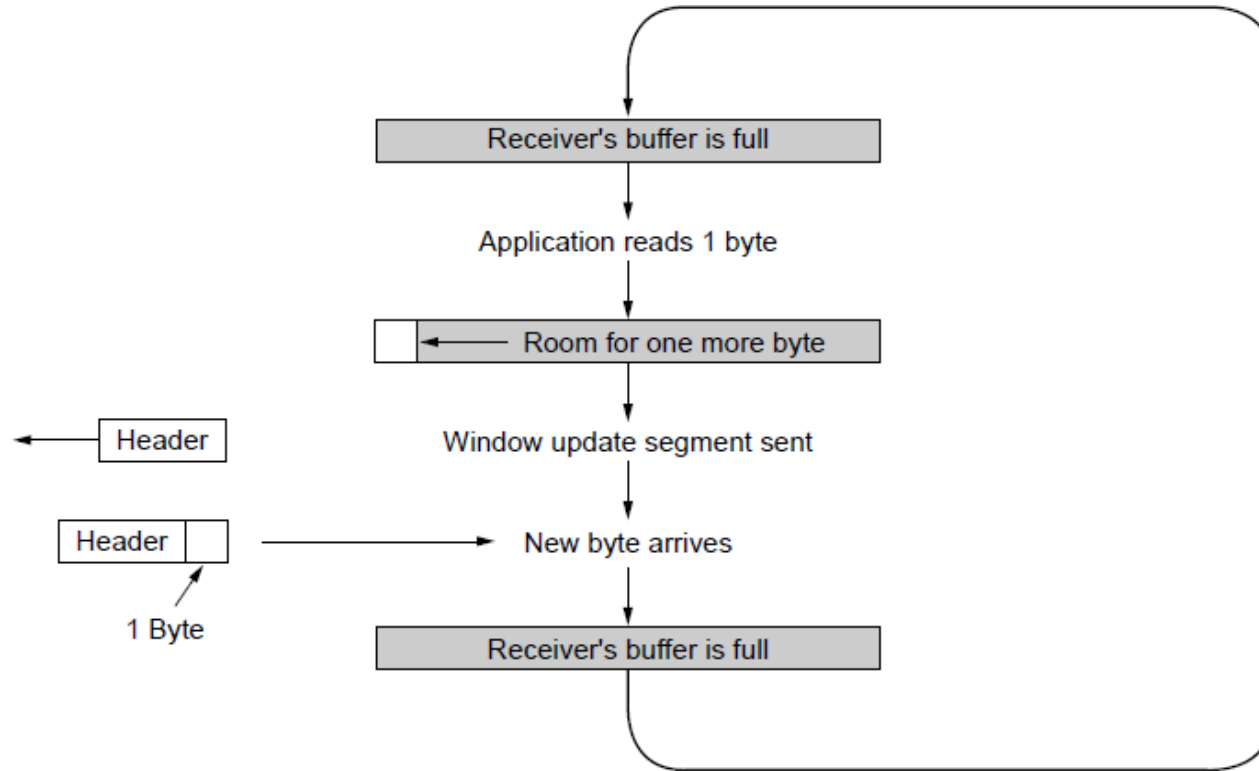
First, urgent data may be sent, for example, to allow the user to kill the process running on the remote machine.

Second, the sender may send a 1-byte segment to force the receiver to reannounce the next byte expected and the window size. This packet is called a *window probe*.

Nagle's Algorithm

- a) Interactive Editor --- Sending 1-byte would involve 162 bytes (40 to send, 40 to ACK, 41 to update, 40 to ack)
- b) Nagle's Algorithm – When data comes into the sender one byte at a time, just send the first byte and buffer the rest until the outstanding byte is acknowledged

TCP Sliding Window (2)



Silly window syndrome ... Clark's solution

- prevent receiver from sending a window update for 1 byte.

Specifically the receiver should not send a window update until it gets 1000 maximum segment advertised free

Initially, the TCP buffer on the receiving side is full (i.e., it has a window of size 0) and the sender knows this. Then the interactive application reads one character from the TCP stream.

This action makes the receiving TCP happy, so it sends a window update to the sender saying that it is all right to send 1 byte.

The sender obliges and sends 1 byte. The buffer is now full, so the receiver acknowledges the 1-byte segment and sets the window to 0.

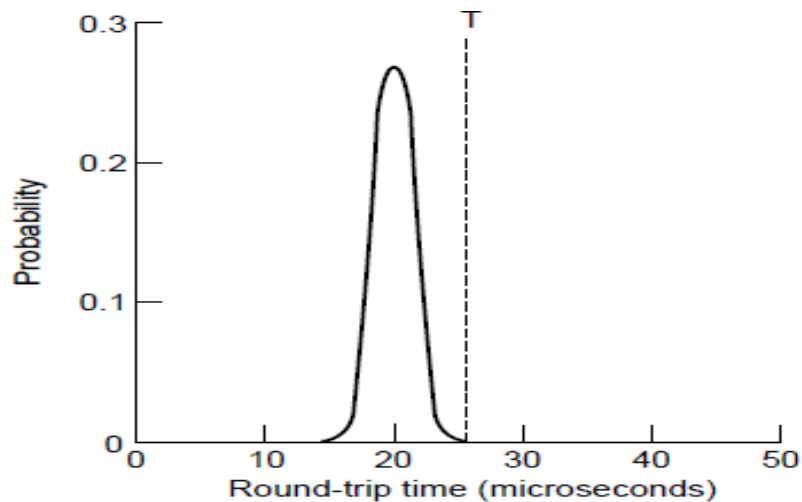
This behavior can go on forever.

Clark's solution is to prevent the receiver from sending a window update for 1 byte. Instead, it is forced to wait until it has a decent amount of space available and advertise that instead.

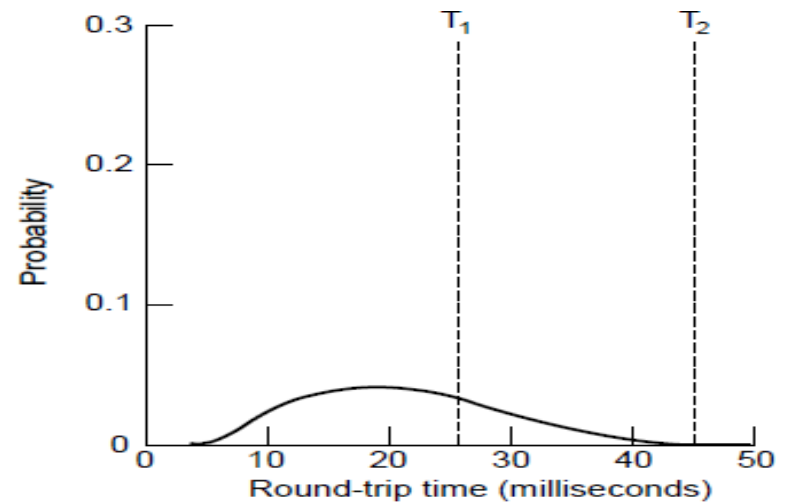
- ✓ Nagle's algorithm and Clark's solution to the silly window syndrome are complementary.
- ✓ Nagle was trying to solve the problem caused by the sending application delivering data to TCP a byte at a time.
- ✓ Clark was trying to solve the problem of the receiving application extracting the data up from TCP a byte at a time.
- ✓ The goal is for the sender not to send small segments and the receiver not to ask for them.

TCP Timer Management

(a). Retransmission timer



(a)



(b)

a) Probability density of acknowledgement arrival times in the data link layer.

(b) Probability density of acknowledgement arrival times for TCP.

With this drawback using dynamic algorithm that constantly adapts the timeout interval is used.

For each connection, TCP maintains a variable, SRTT (Smoothed Round-Trip Time), that is the best current estimate of the round-trip time to the destination.

The acknowledgement gets back before the timer expires, TCP measures how long the acknowledgement took, say, R . It then updates SRTT according to the formula

$$\text{SRTT} = \alpha \text{SRTT} + (1 - \alpha) R$$

where α is a smoothing factor that determines how quickly the old values are forgotten. Typically, $\alpha = 7/8$. This kind of formula is an EWMA (Exponentially Weighted Moving Average) or low-pass filter that discards noise in the samples.

Initial implementations of TCP used $2xRTT$, but experience showed that a constant value was too inflexible because it failed to respond when the variance went up.

Jacobson proposed making the timeout value sensitive to the variance in round-trip times as well as the smoothed round-trip time

Retransmission Timer

RTTVAR (Round Trip Time VARIation) that is updated using the formula

$$\mathbf{RTTVAR = \beta RTTVAR + (1 - \beta) \quad |SRTT - R|}$$

This is an EWMA as before, and typically $\beta=3/4$.

The retransmission timeout, RTO, is set to be

$$\mathbf{RTO = SRTT + 4 \times RTTVAR}$$

Retransmission Timer

- ✓ In dynamic estimation of RTT when a segment times out and is resent. It is not clear whether the acknowledgement is from the original or resend.
- ✓ Karn's algorithm – Proposed --Don't include resend packets RTT into calculation.
- ✓ Timeout is doubled on each successive retransmission until the segments get through the first time.

Persistence Timer

- ✓ It is designed to prevent the following deadlock.
- ✓ The receiver sends an acknowledgement with a window size of 0, telling the sender to wait. Later, the receiver updates the window, but the packet with the update is lost. Now the sender and the receiver are each waiting for the other to do something.
- ✓ When the persistence timer goes off, the sender transmits a probe to the receiver. The response to the probe gives the window size.
- ✓ If it is still 0, the persistence timer is set again and the cycle repeats.
- ✓ If it is nonzero, data can now be sent.

Keepalive timer

- ✓ When a connection has been idle for a long time, the keepalive timer may go off to cause one side to check whether the other side is still there.
- ✓ If it fails to respond, the connection is terminated.
- ✓ It is controversial because it adds overhead and may terminate healthy connection due to a transient network partition

- ✓ The last timer used on each TCP connection is the one used in the *TIME WAIT* state while closing.
- ✓ It runs for twice the maximum packet lifetime to make sure that when a connection is closed, all packets created by it have died off.

TCP Congestion Control

- ✓ One of the key functions of TCP for last: congestion control. When the load offered to any network is more than it can handle, congestion builds up.
- ✓ Transport protocol using an AIMD (Additive Increase Multiplicative Decrease) control law in response to binary congestion signals from the network would converge to a fair and efficient bandwidth allocation

Give formula

Desirable Bandwidth Allocation

- ✓ A good allocation will deliver good performance because it uses all the available bandwidth but avoids congestion, it will be fair across competing transport entities, and it will quickly track changes in traffic demands.
- ✓ An efficient allocation of bandwidth across transport entities will use all of the network capacity that is available.
- ✓ If there is a 100-Mbps link, five transport entities should get 20 Mbps each. They should usually get less than 20 Mbps for good performance.
- ✓ **goodput** (or rate of useful packets arriving at the receiver) as a function of the offered load.

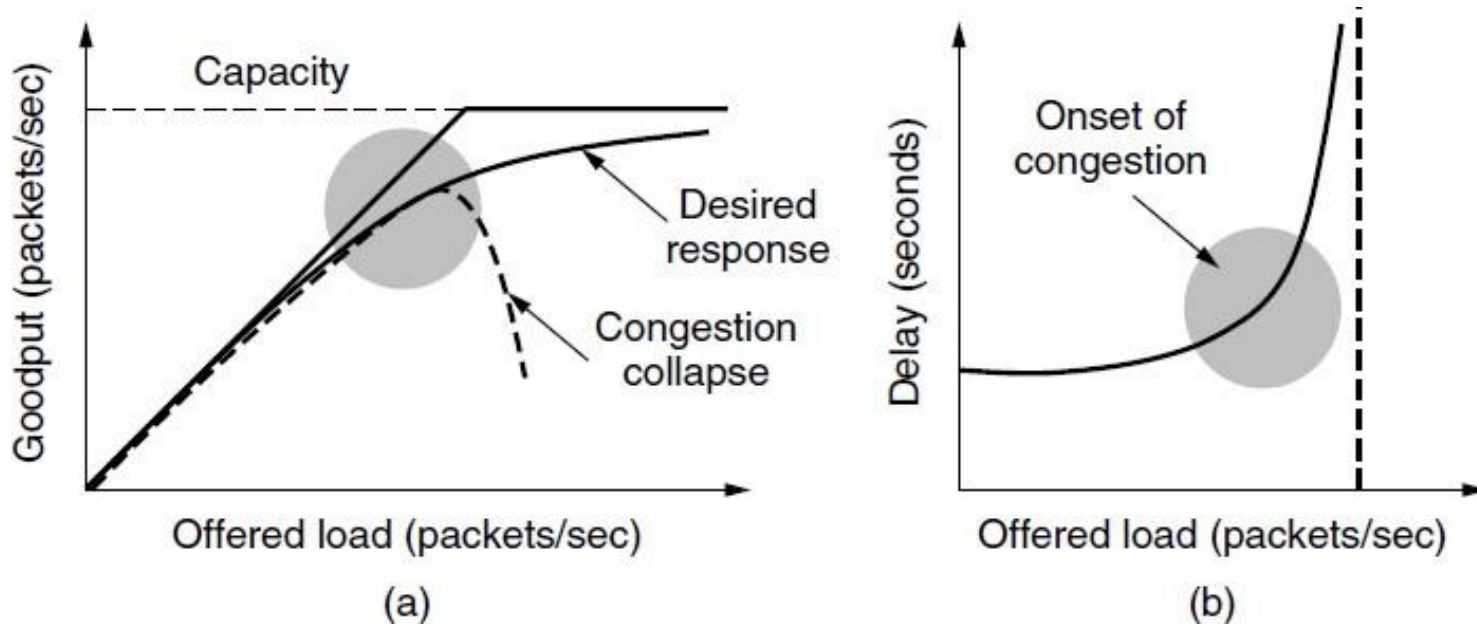


Figure 6-19. (a) Goodput and (b) delay as a function of offered load.

As the load approaches the capacity, the delay rises, slowly at first and then much more rapidly. This is again because of bursts of traffic that tend to mound up at high load.

$$\textit{Power} = \frac{\textit{load}}{\textit{delay}}$$

Power will initially rise with offered load, as delay remains small and roughly constant, but will reach a maximum and fall as delay grows rapidly. The load with the highest power represents an efficient load for the transport entity to place on the network.

Max-Min fairness.

- ✓ This form of fairness that is often desired for network.
- ✓ An allocation is max-min fair if the bandwidth given to one flow cannot be increased without decreasing the bandwidth given to another flow with an allocation that is no larger.
- ✓ Increasing the bandwidth of a flow will only make the situation worse for flows that are less well off.

Max-Min fairness.

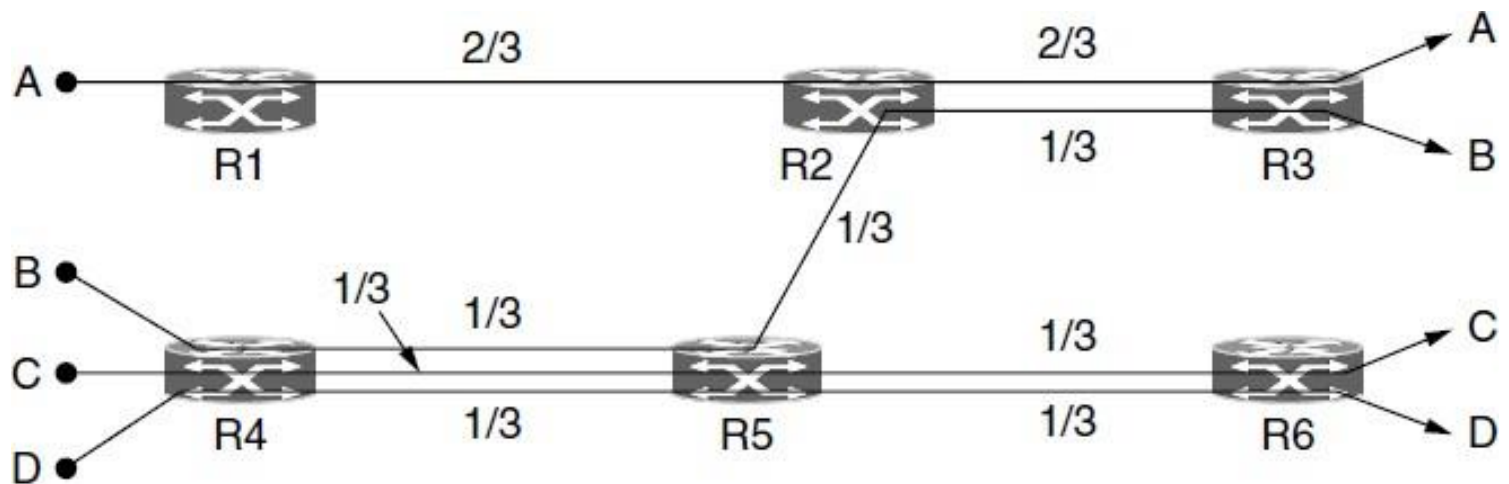


Figure 6-20. Max-min bandwidth allocation for four flows.

Max-Min fairness.

- ✓ No connection be starved of bandwidth than that all connections get precisely the same amount of bandwidth.
- ✓ TCP it is possible to open multiple connections and compete for bandwidth more aggressively.

Convergence

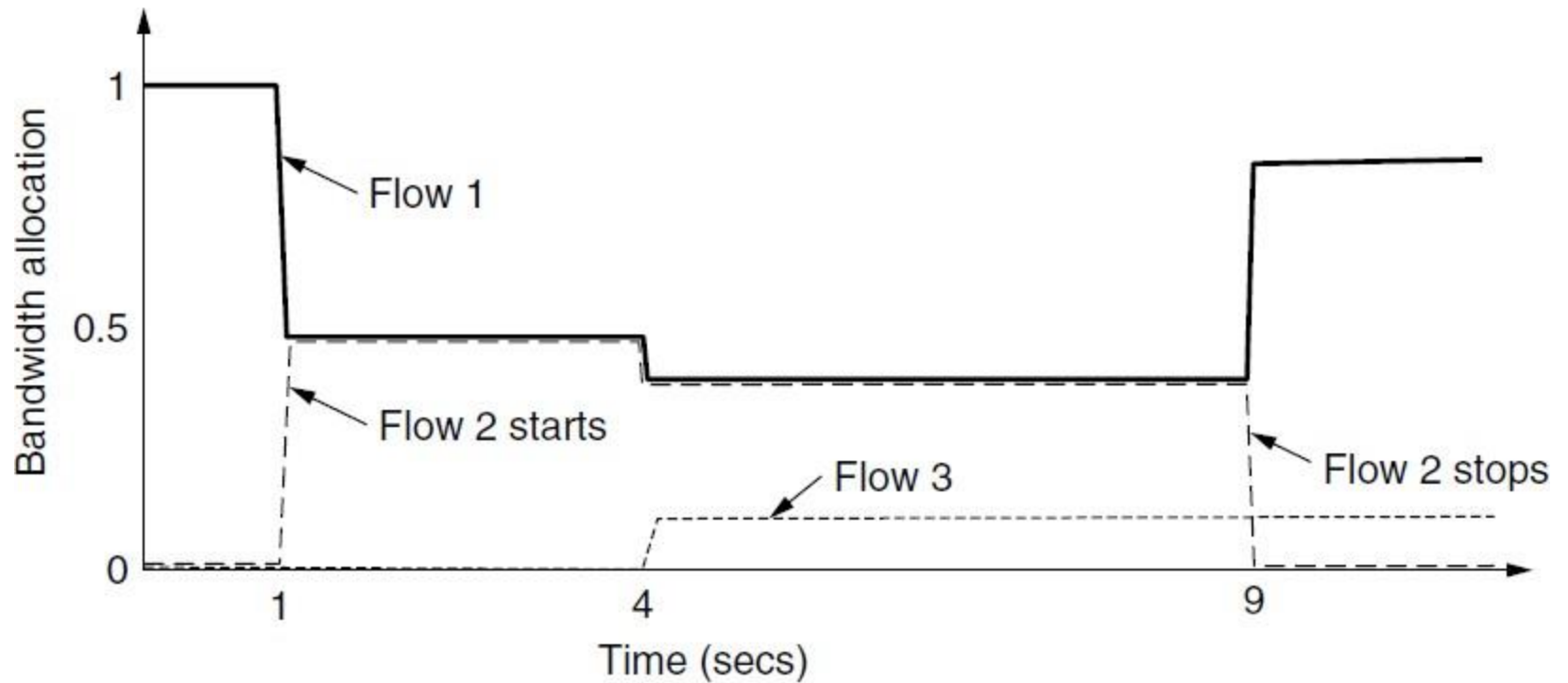


Figure 6-21. Changing bandwidth allocation over time.

Regulating the Sending Rate

- ✓ The sending rate may be limited by two factors.
- ✓ The first is flow control, in the case that there is insufficient buffering at the receiver.
- ✓ The second is congestion, in the case that there is insufficient capacity in the network.

Regulating the Sending Rate

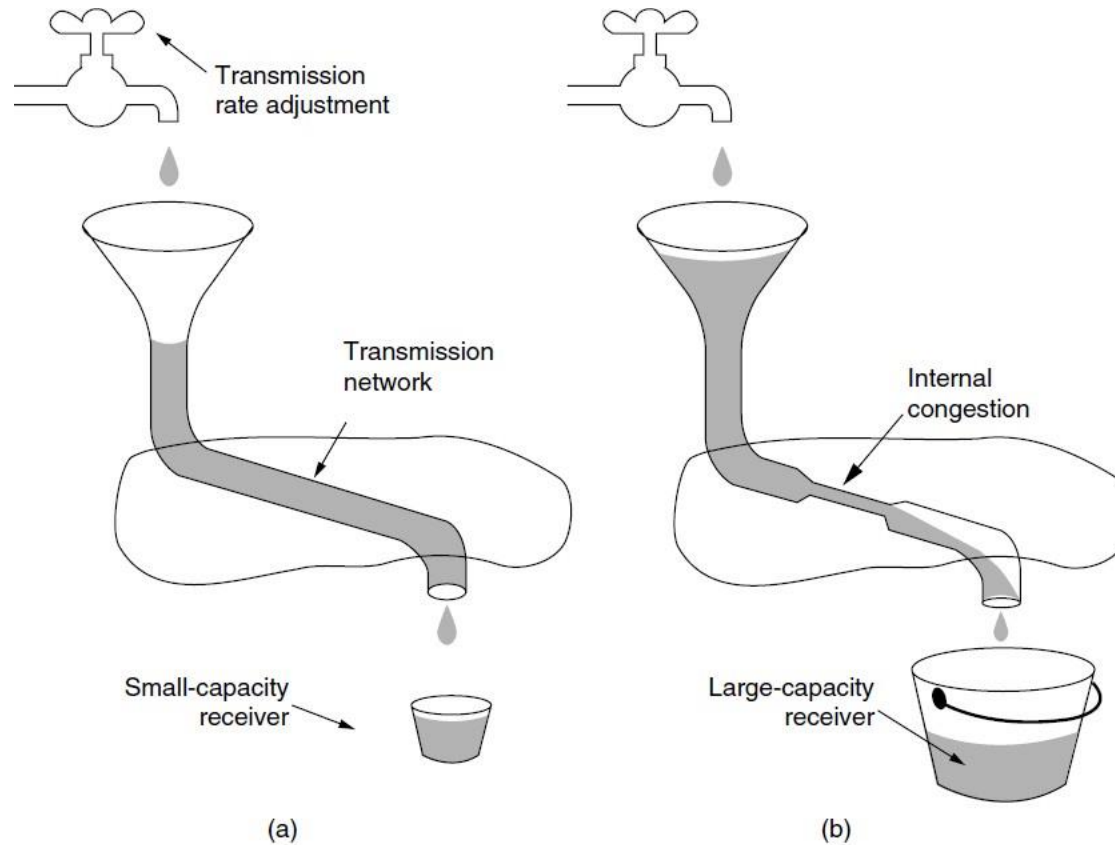


Figure 6-22. (a) A fast network feeding a low-capacity receiver. (b) A slow network feeding a high-capacity receiver.

Regulating the Sending Rate

Protocol	Signal	Explicit?	Precise?
XCP	Rate to use	Yes	Yes
TCP with ECN	Congestion warning	Yes	No
FAST TCP	End-to-end delay	No	Yes
Compound TCP	Packet loss & end-to-end delay	No	Yes
CUBIC TCP	Packet loss	No	No
TCP	Packet loss	No	No

Figure 6-23. Signals of some congestion control protocols.

AIMD (Additive Increase Multiplicative Decrease)

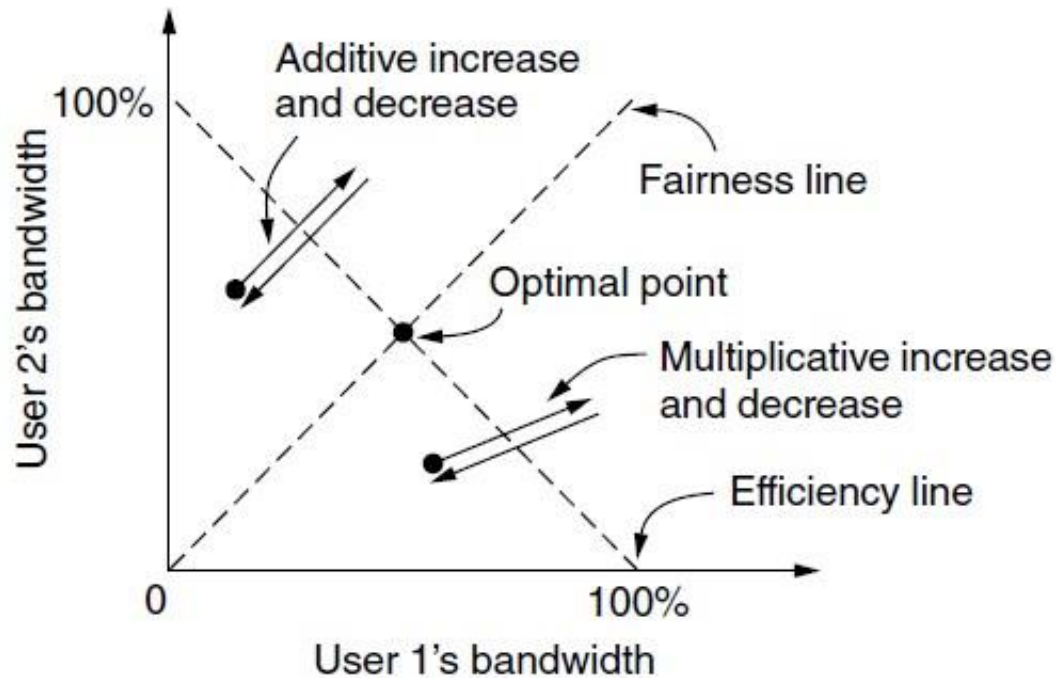


Figure 6-24. Additive and multiplicative bandwidth adjustments.

AIMD (Additive Increase Multiplicative Decrease)

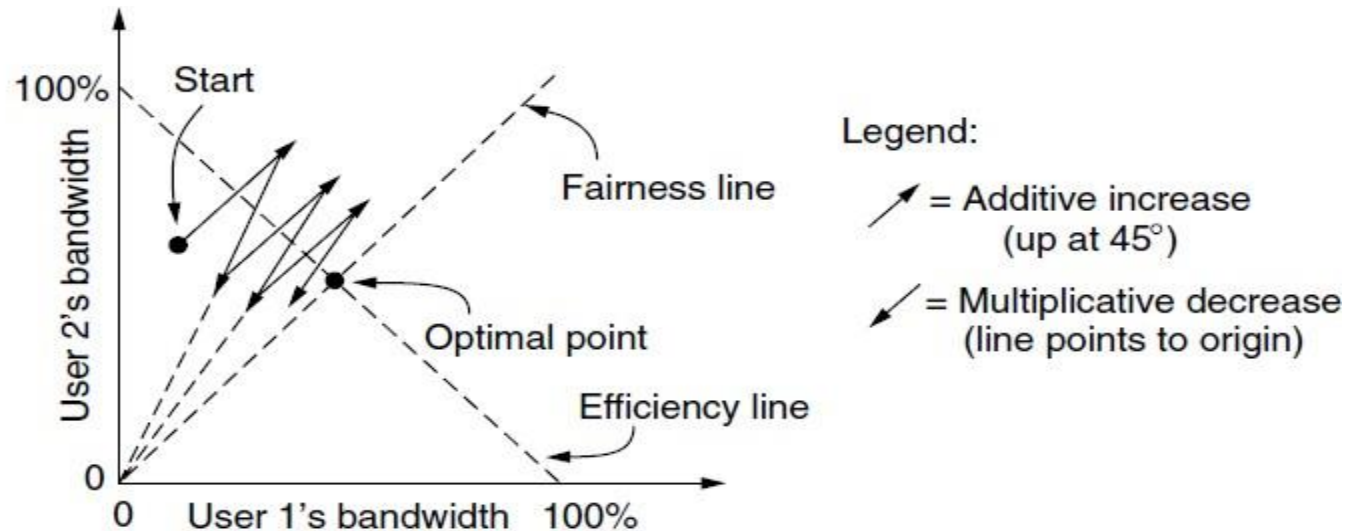


Figure 6-25. Additive Increase Multiplicative Decrease (AIMD) control law.

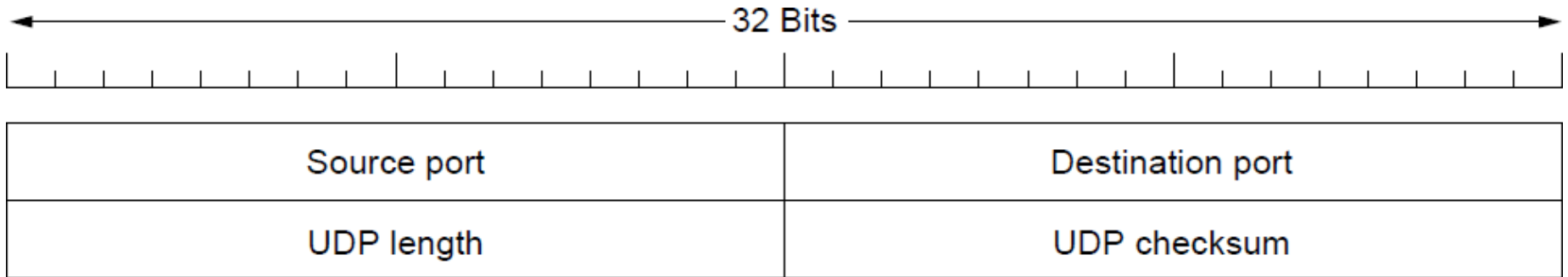
Additively increase their bandwidth allocations and then multiplicatively decrease them when congestion is signaled.

- ✓ Instead of, adjusting the rate directly, a strategy that is often used in practice is to adjust the size of a sliding window.
- ✓ TCP uses this strategy. If the window size is W and the round-trip time is RTT , the equivalent rate is W/RTT .
- ✓ TCP is the dominant form of congestion control in the Internet, there is significant community pressure for new transport protocols to be designed so that they compete fairly with it.

The Internet Transport Protocols: UDP

- Introduction to UDP
- Remote Procedure Call
- Real-Time Transport

Introduction to UDP (1)



The UDP header.

Checksum not used (digitized speech)

Does not flow-control, error control or timing Does Multiplexing

UDP useful – client-server situations, client sends a short request to the server and expects a short reply back. If time-out retransmit rather than establish connection

Introduction to UDP (2)

The *source port* is primarily needed when a reply must be sent back to the source. By copying the Source port field from the incoming segment into the Destination port field of the outgoing segment.

The UDP length field includes the 8-byte header and the data. The minimum length is 8 bytes, to cover the header. The maximum length is 65,515 bytes.

It checksums the header, the data, and a conceptual IP pseudoheader. When performing this computation, the Checksum field is set to zero and the data field is padded out with an additional zero byte if its length is an odd number

TCP Congestion Control

- ✓ All we need to do is to track the congestion window, using sequence and acknowledgement numbers, and adjust the congestion window using an AIMD rule.

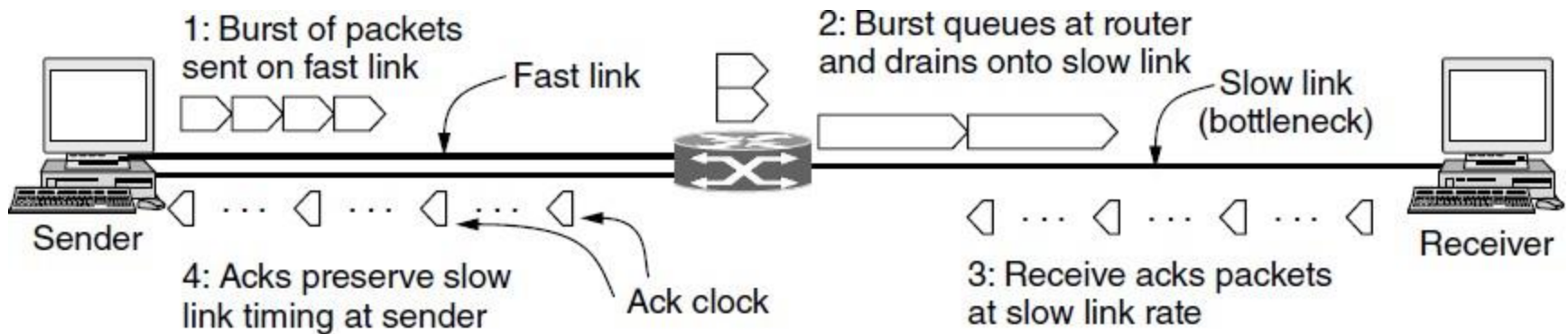


Figure 6-43. A burst of packets from a sender and the returning ack clock.

- ✓ The acknowledgements return to the sender at about the rate that packets can be sent over the slowest link in the path.
- ✓ If Sender injects new packets into the network at this rate, they will be sent as fast as the slow link permits, but they will not queue up and congest any router along the path. This timing is known as an **ack clock**.
- ✓ long time to wait just to get to the right speed for a transfer with ADIMD.

TCP Congestion Control Policy

1. Slow Start Phase: starts slowly increment is exponential to threshold
2. Congestion Avoidance Phase: After reaching the threshold increment is by 1
3. Congestion Detection Phase: Sender goes back to Slow start phase or Congestion avoidance phase.

TCP Congestion Control Policy

Step :1

Slow Start Phase : exponential increment – In this phase after every RTT the congestion window size increments exponentially.

Initially $\text{cwnd} = 1$

After 1 RTT, $\text{cwnd} = 2^1 = 2$ 2 RTT, $\text{cwnd} = 2^2 = 4$

3 RTT, $\text{cwnd} = 2^3 = 8$

TCP Congestion Control Policy

Step 2:

Congestion Avoidance Phase : additive increment – This phase starts after the threshold value also denoted as *ssthresh*. The size of *cwnd*(congestion window) increases additive. After each RTT $cwnd = cwnd + 1$.

Initially $cwnd = i$

After 1 RTT, $cwnd = i+1$ 2 RTT, $cwnd = i+2$ 3 RTT, $cwnd = i+3$

TCP Congestion Control Policy

Step 3:-

Congestion Detection Phase : multiplicative decrement – If congestion occurs, the congestion window size is decreased. The only way a sender can guess that congestion has occurred is the need to retransmit a segment. Retransmission is needed to recover a missing packet which is assumed to have been dropped by a router due to congestion.

Case 1 : Retransmission due to Timeout

Case 2 : Retransmission due to 3 Acknowledgement Duplicates

TCP Slow Start

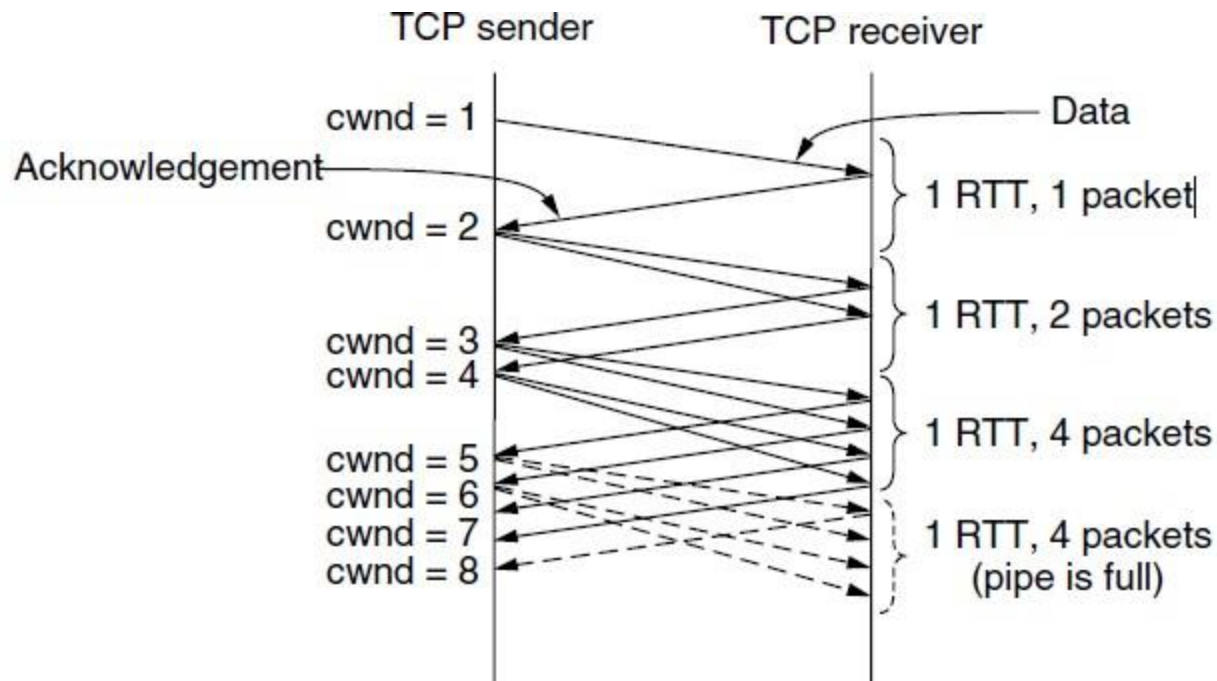


Figure 6-44. Slow start from an initial congestion window of one segment.

TCP Slow Start

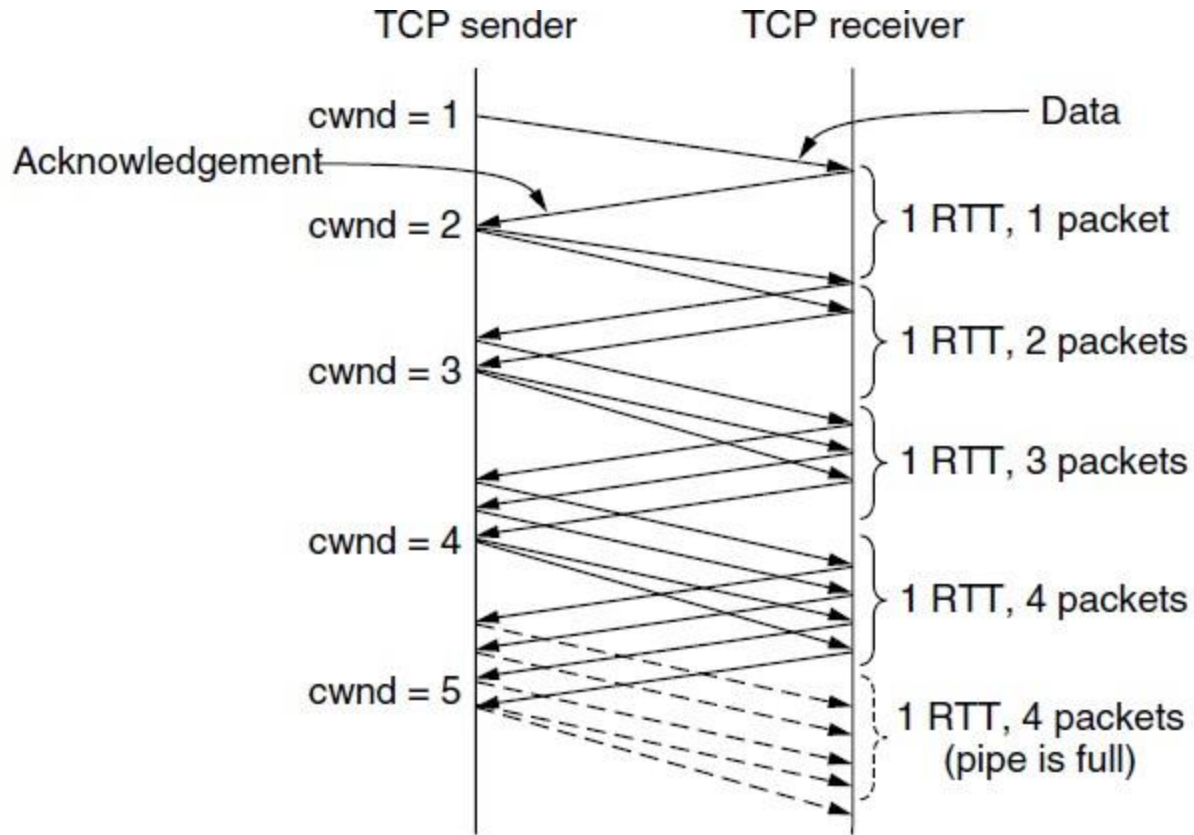


Figure 6-45. Additive increase from an initial congestion window of one segment.

TCP Congestion Control (4)

